

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to pass this in a weekend	1
PART A · FOUNDATIONS AND DATA	4
The Big Picture	5
Number Systems and Conversions	10
Signed Numbers	14
Floating Point, IEEE 754	18
Image and Memory Sizing	22
PART B · THE MACHINE	25
Instruction Sets and Formats	26
Addressing Modes	30
CPU Performance	34
Pipelining and Hazards	38
Memory and Cache	44
PART C · PROVE IT	49
RISC, CISC and the Rest	50
The MCQ Bank, Every Question Answered	52
The 2025 to 2026 Paper, Your Final Mock	64

START HERE

How to pass this in a weekend

You have not read this course and the exam is on Tuesday. That is fine. This book is built for exactly that. It ignores everything the exam will not ask and teaches you, from zero, the dozen skills that answer every question. Do what it says and you will walk in ready.

TEACH

THE ONE IDEA BEHIND THIS BOOK

Computer Architecture looks huge but the exam is small. It reuses about **a dozen ideas**, over and over, with the numbers changed. So this book does not try to cover the syllabus. It works **backwards from the exam**: it takes the real questions as the target, then teaches every skill you need to reach them, starting from nothing. Each skill is taught so you understand *why* it works, not just what to write, which is the only way to stay safe when the examiner changes the numbers on you.

Reading feels like studying and teaches you almost nothing. What puts a skill into your hands is **doing it with the answer covered**. So all through this book you will meet two things, and you must actually use them:

- ▶ A **Your turn** box: a question with new numbers and the answer hidden under a dashed line. Cover it, work it on paper, then check. This is where the learning happens.
- ▶ After a worked example, **Now redo it**: close the book and reproduce the whole solution on blank paper. If you can do that, you own the skill.

The rule: never move on from a skill until you can do its Your-turn box cold, from memory, with different numbers. That single habit is the difference between a pass and a blank page.

Two kinds of question, and this book prepares you for both:

- ▶ **Objective questions (MCQs).** Multiple choice, one right answer out of four. The lecturer has said these are coming. They test definitions and quick judgement across the whole course. Every one from the practice tests is answered and explained in Part C.
- ▶ **The written paper.** The classic shape is **Question One, compulsory, 30 marks**, then **any two of the remaining questions** at 20 marks each, for 70 marks in 2 hours. Question One is where the number-crunching lives, so master it and you have half the paper before you choose anything. The whole 2025/2026 paper is worked at the back as your final mock.

Three days, three sittings. Do the Your-turn boxes as you go, do not just read.

- ▶ **Saturday, Part A.** The big picture, then all the number skills: number systems, signed numbers, floating point, and image sizing. These are pure marks and most of the numeric MCQs.
- ▶ **Sunday, Part B.** The machine: instruction formats, addressing modes, CPU performance, pipelining, and memory and cache. This is the heart of Question One.
- ▶ **Monday, Part C.** The last of the theory, then work the whole **MCQ bank** as a test, then sit the **2025/2026 paper** under the clock. Spend the evening redoing anything you got wrong.

If you are short on time, the order is chosen so the earliest sections are the highest value. Never skip Part A: everything else stands on it.

The boxes, and what each is for

Navigate by the colour and label of these boxes.

TEACH

A concept, from zero, in plain words. Blue.

● MUST DO

METHOD

The few things to know by heart: a rule, or a step-by-step **method** you apply to any numbers. This violet is reserved, so wherever you see it, memorise it.

WORKED EXAMPLE

STEP BY STEP

One full solution, a step at a time, with the reason on every line. Amber. Follow it, then use the **Now redo it** line to reproduce it from memory.

YOUR TURN

What should you do the instant you reach one of these?

ANSWER

Cover the answer, work it on paper with the numbers given, then check. The whole point is retrieval, not reading.

⚠ TRAP

The exact mistake students make here every year. Read these twice; this is the easiest place to lose marks you did not need to.

PART A

Foundations and Data

The bedrock first, then every number skill the exam leans on. Nothing here assumes you know anything. Do it in order: each section stands on the one before.

1010 0011 = A3 = 163

binary

hex

decimal

The Big Picture

Before any sums, the shape of the whole thing. Half the objective questions live right here: what a computer is made of, and the one distinction the examiner asks in a dozen disguises, architecture versus organization.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 A computer runs a program by repeating three steps forever: it **fetches** an instruction from memory, **decodes** it (works out what it means), then **executes** it, and loops back to do it all again. That never-ending loop is the **fetch, decode, execute cycle**. **Name the three steps, in order.**
- 2 ✓ **fetch, decode, execute.** Where do those instructions live? In **memory**, written as plain binary numbers, sitting right beside the **data** they work on. This one idea, the program and the data together in the **same** memory, is the **stored-program** (or **Von Neumann**) concept. **Are the program and the data kept in the same memory, or in two separate ones?**
- 3 ✓ **the same memory.** That is exactly what makes a computer **general**: change the numbers in memory and the same machine does a different job. Now look inside the processor, the **CPU**. Its first working part is the **control unit (CU)**, the conductor: it fetches each instruction and sends out the **control signals** that make everything act at the right moment. **Does the CU itself do the adding and subtracting, yes or no?**
- 4 ✓ **no.** The adding, subtracting, AND, OR and comparing is the job of the second part, the **arithmetic and logic unit (ALU)**. So keep the two apart: the **CU directs**, the **ALU calculates**. **Which unit actually works out $7 + 5$?**
- 5 ✓ **the ALU.** The CPU also keeps a few tiny, very fast stores called **registers**, each holding something it needs right now. The first is the **program counter (PC)**: it holds the **address of the next instruction** to fetch. **What does the PC hold?**
- 6 ✓ **the address of the next instruction.** Once an instruction has been fetched, it is held in the **instruction register (IR)** while it is decoded and run. **Which register holds the instruction currently being executed?**
- 7 ✓ **the IR.** Two more registers, both for talking to memory. The **memory address register (MAR)** holds the address the CPU is about to read from or write to. **Does the MAR hold an address, or a value?**
- 8 ✓ **an address.** Its partner, the **memory data register (MDR)**, holds the **data itself** going to or coming from memory. So the MAR says *which location*, and the MDR carries *the value there*. **From memory, name all four registers you have now met.**
- 9 ✓ **PC, IR, MAR, MDR.** How do these parts reach each other and memory? Over three shared sets of wires called **buses**. The **address bus** carries **which location** is wanted. **Which register's contents would travel out on the address bus?**
- 10 ✓ **the MAR** (the address it is holding). The **data bus** carries the **value itself**, and the **control bus** carries the signals that say **read or write, and when**. **Which of the three buses carries the value itself?**

- 11 ✓ **the data bus.** Now the idea the examiner asks in a dozen disguises: **architecture** versus **organization**. **Computer architecture** is **what the machine does, as the programmer sees it**; it answers "*what does the computer do?*" **Does architecture describe what the programmer can see, or the hidden hardware?**
-
- 12 ✓ **what the programmer can see.** Concretely, architecture is the **instruction set architecture (ISA)**: the instructions available, the data types, the addressing modes, the registers a program can name, the instruction formats. Rule of thumb: **if a program can see it or name it, it is architecture**. **So is the list of addressing modes architecture?**
-
- 13 ✓ **yes** (a program chooses among them, so architecture). Its opposite is **computer organization: how the machine is built to do it**. It is the hidden hardware, the control signals, the buses, the memory technology, the cache, how many execution units there are; the programmer never sees it, and it decides how fast the job gets done. **Architecture answers "what"; organization answers which one-word question?**
-
- 14 ✓ **"how"**. Here is the clean test: **can a program see it or name it? Then architecture. Is it an implementation choice hidden inside? Then organization.** Try it on a hard one, **the size of the cache**. **Is cache size architecture or organization?**
-
- 15 ✓ **organization** (a hidden build choice; no program names the cache size). Because the ISA is all a program sees, two chips with the **same ISA** run the **same programs** (they are **binary compatible**), yet can differ wildly in speed because their organization differs; that deeper hardware realisation is the **microarchitecture**. **If two chips share an ISA, will one compiled program run on both?**
-
- 16 ✓ **yes**. One hook to lock it in for the test: architecture is the **blueprint** (the design); organization is the **construction** (how it is actually built). Same blueprint, two builders, two speeds. (There are two whole ISA styles you will meet in Part C: the reduced instruction set computer, RISC, and the complex instruction set computer, CISC.) Now slide down to the sorting card below and read both columns; most arch-vs-org questions are simply "*which side is this?*"

Memorise these two lists. Most arch-vs-org MCQs are just "which side is this?"

ARCHITECTURE (PROGRAMMER-VISIBLE, "WHAT")	ORGANIZATION (HIDDEN BUILD, "HOW")
Instruction set and instruction formats	Control signals and control-unit design
Addressing modes	Buses and interconnection
Data types and their representation	The memory technology used
Registers a program can name	Cache size and cache implementation
Whether floating-point instructions exist	Number of cores and execution units, clock circuitry

Memory hook: architecture is the **blueprint** (the design); organization is the **construction** (how it is actually built). Same blueprint, two builders, two costs and two speeds.

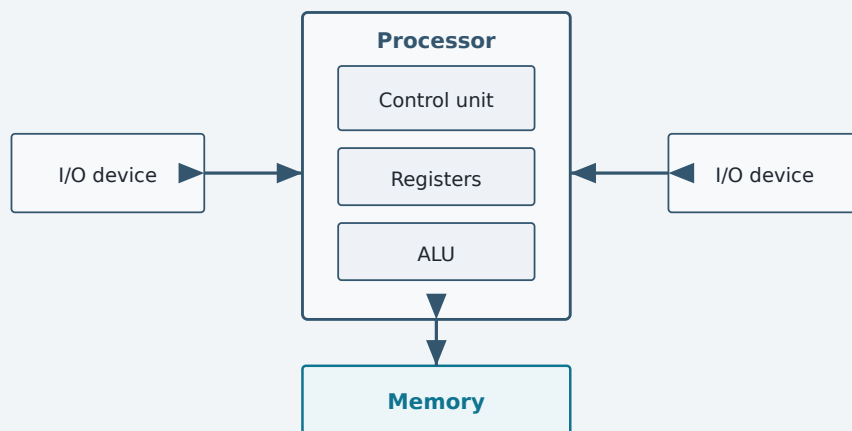
WORK IT FRAME BY FRAME

THE TWO MEMORY DESIGNS

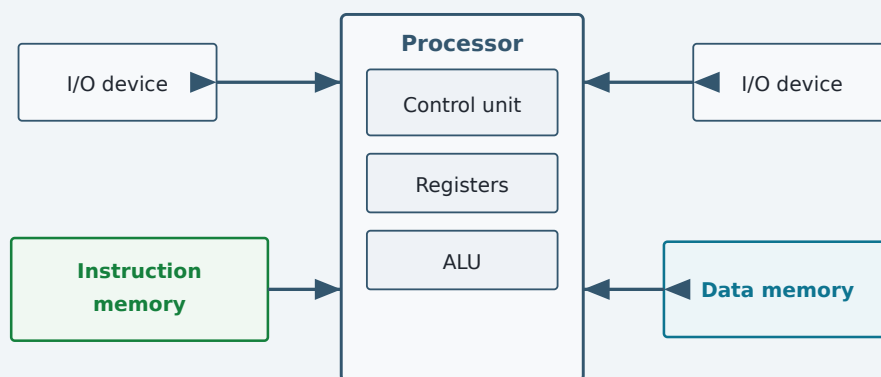
Same as before: keep the page covered, read a frame, do the small task, then slide the card down one line to check.

- Two ways to wire memory to the CPU, and the examiner wants the difference. The first you have already met, **Von Neumann**: **one** memory and one bus shared by **both** instructions and data. It is simple and cheap. **In a Von Neumann machine, how many memories hold the instructions and the data?**
- ✓ **one** (shared). That single shared path has a price: the CPU **cannot** fetch an instruction and read data at the very same moment, they must take turns on the one bus. That traffic jam is the well-known **bottleneck**. **What do we call the slow-down caused by sharing one path?**
- ✓ **the bottleneck**. The rival design is **Harvard**: **separate** memories and **separate** buses, one set for instructions and another for data. It costs more hardware, since the address, data and control lines are duplicated. **How many separate memories does a Harvard machine use?**
- ✓ **two**. That extra hardware buys one thing: Harvard can **fetch an instruction and read data at the same time**, a form of parallelism, and that is its whole advantage. **Which design, Von Neumann or Harvard, can fetch an instruction and read data at once?**
- ✓ **Harvard**. So hold the trade in your head: Von Neumann is **one** memory, cheap but with a shared bottleneck; Harvard is **two** memories, more hardware but faster because both happen together. Now look just below at the two diagrams and the **how to draw each** note, and practise drawing both, because the paper asks for exactly that.

How to draw each (the paper asks for a diagram, so practise it): draw the CPU as one box. For **Von Neumann**, join it by a single line, the shared bus, to **one** Memory box holding instructions and data together. For **Harvard**, draw **two** memory boxes, an instruction memory and a data memory, and join **each** to the CPU by its own bus.



Von Neumann: one Memory, one shared path. One box, one bus.



Harvard: a separate instruction memory and data memory. Two boxes, two buses.

How to remember it: Von Neumann = **one** box, **one** bus. Harvard = **two** boxes, **two** buses; picture the two upright strokes of the letter **H** as the two separate memories.

YOUR TURN

SORT FIVE THINGS

Label each as architecture (A) or organization (O): (1) the list of addressing modes, (2) the size of the cache, (3) the number of general registers a program can use, (4) the technology the main memory is built from, (5) the control signals the control unit sends.

ANSWER

1 A (programmer names them), 2 O (hidden size choice), 3 A (programmer-visible), 4 O (implementation), 5 O (hidden). If you can explain *why* for each, you have this topic.

A bank upgrades its servers to processors with larger caches and faster RAM while keeping the same x86 instruction set. What has primarily changed? (a) Architecture only (b) Organization only (c) Both (d) Neither

ANSWER

(b) Organization only. Same instruction set means the **architecture is unchanged**; larger caches and faster RAM are **organization**. This exact shape, "same ISA, faster hardware", appears again and again: the answer is always organization.

A developer runs the same compiled application on two processors from different manufacturers. This is possible because both implement the same: (a) Instruction Set Architecture (b) Microarchitecture (c) Control unit design (d) Cache hierarchy

ANSWER

(a) Instruction Set Architecture. The same binary program runs on both because they share an **ISA**. Their microarchitecture, control unit and cache may all differ; that is organization, and it only changes speed, not what runs.

A processor fetches an instruction while simultaneously reading data from memory. Which architecture makes this possible? (a) Stack (b) CISC (c) Von Neumann (d) Harvard

ANSWER

(d) Harvard. Separate instruction and data memories and buses let both happen at once. Von Neumann shares one path, so it cannot.

⚠ TRAP

When an MCQ says "**same instruction set**" or "**same ISA**" but the hardware changed, the change is **organization**, never architecture. And "which is visible to the programmer?" almost always points to the **instruction formats / addressing modes / registers** (architecture), not to caches, buses or memory technology (organization).

FOUNDATIONS AND DATA

Number Systems and Conversions

*Everything a computer holds is binary. This skill is turning numbers between binary, decimal and hexadecimal without thinking, plus the special decimal codes, and it is easy marks. We build it as a **programme**: many tiny steps worked through one at a time, and you check each one before the next. Go slowly and it will soon feel automatic.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 A **base** just means how many symbols you count with before you start a new column. Decimal is base 10: the ten symbols 0 to 9. A computer wire is only ever on or off. **So how many symbols does binary have?**
- 2 ✓ **2, just 0 and 1** (on or off), which is base 2. Binary is correct but gets very long, so we bundle it up as **hexadecimal**, base 16, whose symbols run 0 to 9 and then A, B, C, D, E, F. **A is the first letter-symbol; what decimal value does the hex digit A stand for?**
- 3 ✓ **10**. Then B = 11, C = 12, D = 13, E = 14, F = 15. Next, **place value**: reading a number from right to left, the columns are worth $\text{base}^0 = 1$, then base^1 , then base^2 , and so on. **In hex, what is the place value of the third column from the right?**
- 4 ✓ **$16^2 = 256$** . So the hex columns are 1, 16, 256, 4096. Three size words before we convert: one binary digit is a **bit**, four bits are a **nibble**, and eight bits are a **byte**. **How many bits are in a byte?**
- 5 ✓ **8** (and four bits are a nibble). Now the first conversion. To turn **any base into decimal**, multiply each digit by its place value and add the results. Take the hex number 2F: its columns are 16 and 1, and F = 15. **Work out $2 \times 16 + 15$.**
- 6 ✓ **47, so hex 2F = 47**. That is every base-to-decimal question. The reverse, **decimal to any base**, divides repeatedly by the base, jots each remainder, and reads the remainders bottom to top. For 13 to binary: $13 \div 2 = 6 \text{ r } 1$, $6 \div 2 = 3 \text{ r } 0$, $3 \div 2 = 1 \text{ r } 1$, $1 \div 2 = 0 \text{ r } 1$. **Read the four remainders from the bottom up.**
- 7 ✓ **1101**. Let us **prove it** rather than trust it. The binary columns are 8, 4, 2, 1, and 1101 has 1-bits in the 8, 4 and 1 columns. **Add $8 + 4 + 1$; do you land back on 13?**
- 8 ✓ **yes, $8 + 4 + 1 = 13$** . The conversion checks out. Now **binary to hex**. Because $16 = 2^4$, one hex digit is exactly four bits, so group the bits into fours **from the right** and turn each four into one hex digit. Take 1101 0110: the right group 0110 = $4 + 2 = 6$. **What hex digit is the left group 1101?**
- 9 ✓ **D** (that is $8 + 4 + 1 = 13$). So 1101 0110 = D6, and going the other way you simply expand each hex digit back into its four bits. **Read on.**
- 10 A few **powers of two** must be instant. $2^{10} = 1024$, which we call "1 K"; then $2^{20} = 1 \text{ M}$ and $2^{30} = 1 \text{ G}$. **How many bytes is 2^{10} , and what one-letter name do we give it?**
- 11 ✓ **1024 bytes, called 1 K**. Every memory and cache question leans on these. Now the special **decimal codes**, where each decimal digit is kept on its own. First is BCD. **Read on.**

- 12 **BCD** (binary coded decimal), the 8421 code, writes **each decimal digit** as its own 4 bits. So 59 becomes 5 = 0101 and 9 = 1001, written 0101 1001. It is a **weighted** code (the bit positions are worth 8, 4, 2, 1). **Write the single digit 7 in BCD.**
- 13 ✓ **0111** ($4+2+1 = 7$). BCD is just the digit in plain 4-bit binary. The next code, **Excess-3**, takes the digit, **adds 3**, then writes the result in 4 bits. So 9 becomes $9+3 = 12 = 1100$. **Write the digit 6 in Excess-3.**
- 14 ✓ **1001** ($6+3 = 9 = 1001$); to decode you subtract 3. Excess-3 is **non-weighted** (its bits carry no fixed weight) and self-complementing. **Read on.**
- 15 Last, the **character codes**, which store text and not numbers. **ASCII** encodes English letters, digits and symbols in 7 or 8 bits; **Unicode** was introduced to cover the characters of all the world's languages. **Which of the two would you need for Chinese or Arabic letters?**
- 16 ✓ **Unicode** (it covers every language). That is every fact in this skill. Now watch the moves used together in the two worked examples below, then sit the recalls with the answers covered.

● MUST DO

THE THREE CONVERSIONS, AND POWERS OF TWO

- ▶ **Any base to decimal:** multiply each digit by its place value and add. In hex the places are 1, 16, 256, 4096.
- ▶ **Decimal to any base:** divide repeatedly by the base, write down each remainder, then read the remainders **bottom to top**.
- ▶ **Binary to hex:** group the bits into **fours from the right**, turn each group of four into one hex digit. Hex to binary is the reverse: each hex digit becomes four bits. This works because $16 = 2^4$, so one hex digit is exactly four bits.

N	0	1	2	3	4	5	6	7	8	9	10
2^N	1	2	4	8	16	32	64	128	256	512	1024

Memory hook: know these powers of two cold; $2^{10} = 1024 = \text{"1 K"}$, then $2^{20} = 1 \text{ M}$, $2^{30} = 1 \text{ G}$. Every cache and memory question needs them.

WORKED EXAMPLE

HEX TO DECIMAL

Convert the hex number **7D3A** to decimal.

place:	4096	256	16	1
digit:	7	13(D)	3	10(A)
	----	----	---	---
	7 x 4096	= 28 672		
	13 x 256	= 3 328		
	3 x 16	= 48		
	10 x 1	= 10		

		32 058		

$$7D3A = 28\,672 + 3\,328 + 48 + 10 = 32\,058$$

Now redo it: the only hard part is remembering 4096, 256, 16, 1. Write those four first, then multiply.

WORKED EXAMPLE

BINARY TO HEX

Convert binary **11010110** to hex.

- 1 Group in fours from the right: 1101 0110.
- 2 Each group of four to one hex digit: 1101 = 8+4+1 = 13 = **D**; 0110 = 4+2 = **6**.
- 3 Write them in order: **D6**.

$$11010110 = 1101\ 0110 = D6 \text{ (check: both equal 214)}$$

Now redo it: go backwards too. 2A7F to binary is 2=0010, A=1010, 7=0111, F=1111, giving 0010101001111111.

YOUR TURN

CONVERT THREE

(1) Write decimal 255 in hex. (2) How many hex digits does a 20-bit binary number need? (3) A hex value shifted right by one bit is the same as which operation?

ANSWER

(1) $255 = 15 \times 16 + 15 = \mathbf{FF}$. (2) $20 \div 4 = \mathbf{5}$ hex digits (one hex digit per four bits). (3) shifting right one bit is **dividing by 2** (left one bit is times 2).

YOUR TURN

MCQ, THE CODES

(1) The Excess-3 code for decimal 9 is? (2) The Excess-3 code 0110 is which decimal digit? (3) The 8421 BCD of decimal 59 is?

ANSWER

(1) $9 + 3 = 12 = \mathbf{1100}$. (2) $0110 = 6$, minus 3 = **3**. (3) $5 = 0101$, $9 = 1001$, so **01011001**. Remember: BCD is the digit itself in 4 bits; Excess-3 is the digit plus 3.

⚠ TRAP

Group binary into fours **from the right**, never the left, and pad the leftmost group with leading zeros if it is short. And do not confuse the codes: **BCD is weighted** (8421) and is just the digit in binary; **Excess-3 is non-weighted** and is the digit **plus 3**. "BCD" stands for **binary coded decimal**, nothing else.

FOUNDATIONS AND DATA

Signed Numbers

*This is the most asked numeric topic, and the one most people find confusing. So we do it as a **programme**: many tiny steps, and you check each one before moving on. Go slowly. By the last frame the moves will feel obvious.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Start with 8 bits and **no sign** yet. The patterns run 0000 0000, 0000 0001, all the way up to 1111 1111. That top one, 1111 1111, is $2^8 - 1$. **Write its value in decimal, then slide down.**
- 2 ✓ **255**. Every one of the 256 patterns is a positive number; there is no minus sign anywhere yet. To make room for negatives we sacrifice the **top (leftmost) bit** and let it carry the sign: a top bit of 0 means positive. **So a top bit of 1 must mean what?**
- 3 ✓ **negative**. So just by glancing at 1010 1010 (top bit 1) you know it is a negative number. The only question left in the whole topic is *which* pattern to use for a given negative. The scheme real computers use is called **two's complement**, and the next few frames build it slowly. **Read on.**
- 4 Picture the 8 bits as a car **odometer**. It counts up 0000 0000, 0000 0001, and on to 1111 1111, then rolls back round to 0000 0000. Now turn it **one step backwards** from 0000 0000. **What pattern do you land on?**
- 5 ✓ **1111 1111**. One step below zero. So it is natural to call 1111 1111 the value **-1**. Step back once more, to 1111 1110, and that is **-2**. Negatives are just the patterns you meet counting **down** from the top. **Counting back once more, what pattern is -3?**
- 6 ✓ **1111 1101**. Counting all the way down would be slow, so here is the **shortcut** to build any negative: write the positive, **flip every bit**, then **add 1**. Try it on -1. Start with $+1 = 0000 0001$. **Flip every bit and write the result.**
- 7 ✓ **1111 1110**. Now the second half of the shortcut. **Add 1 to 1111 1110. What do you get?**
- 8 ✓ **1111 1111**. That is the same pattern the odometer gave for -1, so the shortcut is trustworthy. Say it like a chant: **"flip every bit, add one."** Now your turn, all by yourself: make **-5** in 8 bits. Start with $+5 = 0000 0101$. **Flip, then add 1.**
- 9 ✓ **1111 1011**. (0000 0101 flips to 1111 1010, and adding 1 gives 1111 1011.) That pattern is **-5**. Notice you only ever do three things: **positive, flip, add one**. **Read on.**
- 10 Now the other direction: **reading** a negative pattern back into a number. The trick is the very same one. Take 1111 1011 (top bit 1, so it is negative). Flip it and add 1 to find its **size**. **Flip 1111 1011, add 1, and say what size you get.**
- 11 ✓ **0000 0101 = 5**, so the value is **-5**. (Flip 1111 1011 to 0000 0100, add 1 to get 0000 0101.) Writing a negative and reading one use the **same move**, which is why it is worth learning well. **Read on.**

- 12 Why did the world settle on this scheme? Two reasons. It has only **one** zero (just 0000 0000), and the ordinary adder does subtraction for free, because $a - b$ is the same as $a + (-b)$. Here is the proof. Add -5 (1111 1011) to $+5$ (0000 0101), and drop any carry that falls off the top. **What 8-bit pattern is left?**
- 13 ✓ **0000 0000**. (1111 1011 + 0000 0101 = 1 0000 0000; throw away the carry and you have 0000 0000, which is zero.) A number plus its negative gives zero, which is exactly what a negative should do. The scheme really works. **Read on.**
- 14 Last piece: the **range**. Half the patterns are negative and half positive, but zero uses up one positive slot, so there is always one more negative than positive: -2^{n-1} to $2^{n-1} - 1$. In 16 bits that is $-32\,768$ to $32\,767$. **So in 8 bits the range is -128 to what?**
- 15 ✓ **127**. (-2^7 to $2^7 - 1$ is -128 to 127 .) That is two's complement, start to finish. **Read on.**
- 16 Two older schemes you only need to **recognise** for the objective test. **Sign-magnitude**: a sign bit then the plain size, so -42 is 1 then 010 1010 = 1010 1010; its drawback is **two zeros** ($+0$ and -0). **One's complement**: negate by flipping every bit only (no add 1); it needs an **end-around carry** when adding and also has two zeros. Two's complement beat both by adding that 1. **How many zeros does sign-magnitude have?**
- 17 ✓ **two** ($+0$ and -0). That is every idea in this topic. Now watch all the moves used together in the worked example below, then sit it yourself with the answer covered.

● MUST DO

THE MOVES ON ONE CARD

- ▶ **Write a negative**: positive in n bits, **flip every bit, add 1**.
- ▶ **Read a negative** (top bit 1): **flip, add 1** for the size, then put a minus in front.
- ▶ **Add**: add the patterns, **drop** any carry off the top (no end-around carry).
- ▶ **Range in n bits**: -2^{n-1} to $2^{n-1} - 1$ (16 bits: $-32\,768$ to $32\,767$).

Memory hook: one's complement is "flip"; two's complement is "flip and add one".

WORKED EXAMPLE

THE METHOD, VERTICALLY

Store -182 , 487 and -1023 in a 16-bit two's-complement machine (binary and hex) and add $(-182) + 487$. **The exam sets exactly this; you will sit it verbatim in the final mock at the back.** Here is how you write it on paper, one move at a time.

GIVEN 16 bits, two's complement · -182 , 487 , -1023
 FORMULA range = -2^{15} to $2^{15} - 1$ · negative = flip the positive and add 1 · group 4 bits per hex digit

Range: -2^{15} to $2^{15} - 1 = -32768$ to 32767 (all three fit)

$487 = 0000\ 0001\ 1110\ 0111 = 01E7$

-182 : $+182 = 0000\ 0000\ 1011\ 0110$
 invert = $1111\ 1111\ 0100\ 1001$
 $+ 1 = 1111\ 1111\ 0100\ 1010 = FF4A$

-1023 : $+1023 = 0000\ 0011\ 1111\ 1111$
 invert = $1111\ 1100\ 0000\ 0000$
 $+ 1 = 1111\ 1100\ 0000\ 0001 = FC01$

$(-182) + 487$:
 $1111\ 1111\ 0100\ 1010$
 $+ 0000\ 0001\ 1110\ 0111$

 $1\ 0000\ 0001\ 0011\ 0001$ drop the carry off the top
 $0000\ 0001\ 0011\ 0001 = 256+32+16+1 = 305$

Range -32768 to 32767 (all fit); $487 = 01E7$, $-182 = FF4A$, $-1023 = FC01$; $(-182) + 487 = 305$.

Now redo it: positive, invert, add 1, every time. Practise with -75 in 8 bits (answer $1011\ 0101$).

YOUR TURN

READ THREE PATTERNS

In 8 bits: (1) what decimal is $1000\ 0000$ in two's complement? (2) what is the two's complement of $0010\ 1100$? (3) what decimal is $1010\ 1010$ in *sign-magnitude*?

ANSWER

(1) top bit 1 so negative; flip to $0111\ 1111$, add 1 = $1000\ 0000 = 128$, so **-128** . (2) flip $0010\ 1100$ to $1101\ 0011$, add 1 = **$1101\ 0100$** . (3) sign bit 1 (negative), magnitude $010\ 1010 = 42$, so **-42** .

(1) How many representations of zero in sign-magnitude? (2) Which scheme needs an end-around carry? (3) The one's-complement pattern 1110 0011 is which decimal? (4) Why is two's complement used?

ANSWER

(1) **two** (+0 and -0). (2) **one's complement**. (3) top bit 1 so negative; flip 1110 0011 to 0001 1100 = 28, so **-28**. (4) **simpler and faster hardware** (one zero, and add and subtract share one circuit).

⚠ TRAP

Reading a negative two's-complement pattern, do **not** just read the magnitude bits off; you must flip and add 1 first. **Overflow** means the true result is too big or too small for the range, not division by zero. And the end-around carry belongs to **one's** complement only; two's complement simply discards the top carry.

FOUNDATIONS AND DATA

Floating Point, IEEE 754

*One 32-bit pattern has to hold both a tiny number like 0.0000001 and a huge one like 6 billion, and plain whole-number binary cannot stretch that far. The fix is a trick you already know from science class, but people find it fiddly, so we do it as a **programme**: many tiny steps, and you check each one before moving on. Go slowly. By the last frame the three fields will feel obvious, and both exam directions fall out of a single line.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Here is the whole problem in one line. You have **one** box, exactly 32 bits wide, and it must be able to write both a tiny number like **0.0000001** and a giant one like **6,000,000,000**. Plain whole-number binary, the kind you have used so far, cannot reach both ends. **Can plain whole-number binary write both of those in 32 bits? Yes or no?**
- 2 ✓ **No.** It runs out of room. But you already beat this same problem in **decimal**, in science class: to write 6,020,000 compactly you put it as **6.02 × 10⁶**. Hidden in that little form are three separate pieces of information. **Name the three pieces.**
- 3 ✓ **a sign (it is positive, +), a fraction (6.02), and an exponent (6).** That is all scientific notation is. Floating point does **exactly** this, but in **base 2**: a sign, a fraction, and a power of **2** instead of 10. Nothing more is going on. **Read on.**
- 4 So a floating-point number is stored as those same three pieces, now called three **fields**, written left to right: the **sign** (s), then the **exponent** (E), then the **mantissa** M (which is just the fraction, also called the significand). **Which of the three fields sits on the far left?**
- 5 ✓ **the sign.** Now their sizes. **Single precision** is **32 bits**, split as **1** sign bit, then **8** exponent bits. **That leaves how many bits for the mantissa, to reach 32?**
- 6 ✓ **23.** (1 + 8 + 23 = 32.) You only need to **recognise** the bigger size: **double precision** is **64 bits**, split 1 + 11 + 52. Everything from here uses single precision. **Read on.**
- 7 One preparation step, called **normalising**. In decimal you insist on **exactly one non-zero digit before the point** (6.02, never 60.2 or 0.602). In binary you do the same: slide the point until the number reads **1.something × 2^e**. Since the only non-zero binary digit there can be is 1, **what single thing does a normalised binary number always start with?**
- 8 ✓ **"1." (a one, then the point).** Always. And here is the first trick that helps: because it is **always** "1.", actually storing that 1 would waste a bit on something you already know for certain. So it is **left out and just assumed**. This is the famous **hidden 1**. **When you READ a stored pattern back, what must you put in front of the mantissa?**
- 9 ✓ **a "1."** Put the hidden 1 back before you read. And going the other way, when you **write** a number you drop that leading 1 and keep only what follows the point. It costs nothing and buys you one extra bit of precision for free. **Read on.**

- 10 Now the second trick, the one that trips up most beginners: the **bias**. The exponent can be **negative** (for a tiny number like 2^{-3}) or positive (for a big one). Rather than give the exponent its own sign bit, we **add a fixed number, the bias**, so the stored exponent is always a positive count. The bias is **127** for single (1023 for double). It is like storing temperatures as "degrees above -127 ", so a minus sign is never needed. **Read on.**
- 11 So the stored exponent field is the true exponent **plus** 127. To undo that and get the **true** exponent back, you simply **subtract** the bias again. Suppose a pattern has an exponent field of **130**. **What is its true exponent?**
- 12 ✓ **3.** ($130 - 127 = 3$.) That is the whole method on one line, so learn it: **value = $\pm 1.M \times 2^{(\text{exponent field} - 127)}$** . Both exam directions fall straight out of it. **Read on.**
- 13 One last reading skill: turning the mantissa into a number. After the hidden "1.", each mantissa bit is a smaller and smaller fraction: the first bit is worth **1/2**, the next **1/4**, the next **1/8**. So a stored mantissa of 100... reads as **1.1** in binary, which is $1 + 1/2$. **What is $1 + 1/2$ in decimal?**
- 14 ✓ **1.5.** Now decode a whole real pattern, one field at a time. Take sign bit **1**, exponent field **130**, mantissa **100...**. The sign bit 1 means negative, the field 130 gives $e = 3$ (you just did that), and the mantissa gives $1.M = 1.5$. First the power: **what is 2^3 ?**
- 15 ✓ **8.** Now hang the sign and the mantissa on it: $\text{value} = -1.5 \times 2^3 = -1.5 \times 8$. **Finish it: what number is that?**
- 16 ✓ **-12.** You just decoded a floating-point pattern by hand. Now **prove the bias to yourself**: to STORE that same exponent of 3 you would go the other way and do $3 + 127$. **What exponent field would you store?**
- 17 ✓ **130.** The very field you started from, so the bias undoes itself cleanly, going in and coming out. That is every idea in the topic: three fields, normalise to 1.something, hide the leading 1, and shift the exponent by the bias. Now watch all the moves used together in the worked example below (it decodes this exact pattern, C1400000), then sit the two "Your turn" boxes with the answer covered.

- ▶ **Decimal to IEEE (single):** (1) sign bit from the sign. (2) Convert the magnitude to binary. (3) Normalise to **1.something** $\times 2^e$ by moving the point. (4) Exponent field = **e + 127**, in 8 bits. (5) Mantissa = the "something" after the point, padded with zeros to 23 bits (the leading 1 is dropped, it is the hidden 1). (6) Put the 32 bits together and group in fours for hex.
- ▶ **IEEE (hex) to decimal:** (1) write the 32 bits, split as 1 / 8 / 23. (2) Sign bit gives the sign. (3) Real exponent **e = exponent field - 127**. (4) Value = $\pm 1.M$ in binary $\times 2^e$ (remember to put the hidden 1 back).

Memory hook: "sign, exponent, mantissa", and the exponent is always **stored plus the bias, read minus the bias**.

WORKED EXAMPLE

THE METHOD, VERTICALLY

Convert **C1400000** (single precision) to decimal. This is the X the exam sets in Question 4a; you will sit it verbatim in the final mock.

GIVEN C1400000h, single precision
 FORMULA value = $(-1)^{\text{sign}} \times 1.M \times 2^{(\text{exp field} - 127)}$

C1400000 = 1 10000010 100 0000...
 sign 1 -> negative
 exp field 1000 0010 = 130, e = 130 - 127 = 3
 mantissa 1.100... = 1 + 1/2 = 1.5
 value = $-1.5 \times 2^3 = -1.5 \times 8 = -12$

C1400000 = $-1.5 \times 2^3 = -12$

Now redo it: exponent field minus 127, then read 1.M as a binary fraction (first bit a half, next a quarter).

YOUR TURN

ENCODE A CLEAN NUMBER

Encode +6.0 in single precision. (Hint: 6 in binary is 110; normalise it.)

ANSWER

6 = 110 = 1.10×2^2 . Sign = 0. Exponent field = 2 + 127 = 129 = 1000 0001. Mantissa = 100...0 (23 bits). Pattern: 0 10000001 10000000000000000000000 = **40C00000**.
 Check by decoding: $+1.5 \times 2^2 = 6$.

(1) How many exponent bits in single precision? (2) The bias in double precision? (3) How many bits is double precision? (4) A floating-point number is stored as which order of fields?

ANSWER

(1) **8**. (2) **1023**. (3) **64**. (4) sign, then exponent, then mantissa (the exam phrases the answer as "sign, mantissa, exponent", meaning those three parts with the sign first; the true field order on the chip is sign, exponent, mantissa).

⚠ **TRAP**

Do not forget the **bias**: the exponent you compute is **e + 127** going in, and **field – 127** coming out. And the leading 1 is **hidden**, so when you read the mantissa put a "1." in front of it. Single precision has range about $10^{\pm 38}$; double about $10^{\pm 308}$.

FOUNDATIONS AND DATA

Image and Memory Sizing

*A small, reliable question: how much memory a picture needs. One formula and a couple of unit facts, and the marks are yours. We build it as a **programme** of tiny steps, worked through one at a time, so cover the page and check yourself as you go.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 A **bitmap** image (also called **raster**) is a grid of tiny squares, and each square stores its own colour as a few bits. **So a bitmap picture is built out of what?**
- 2 ✓ **pixels**. A **vector** image is different: it stores the shapes as maths (lines and curves), so it can be scaled to any size with no loss of quality. **A logo that must be blown up to billboard size with no blur, bitmap or vector?**
- 3 ✓ **vector** (the maths redraws it crisply at any size). A photograph, with its continuous shades, is stored as a bitmap. **Read on.**
- 4 Now **colour depth**, the number of bits each pixel uses. Monochrome (black or white only) needs 1 bit per pixel; greyscale is often 8 bits; full colour is 24 bits. **How many bits per pixel does a monochrome image use?**
- 5 ✓ **1 bit** (only two states, black or white). Now the size formula. The number of bits in a bitmap is width in pixels $\times$ height in pixels $\times$ bits per pixel. Picture a small 100 by 50 pixel monochrome image. **Work out $100 \times 50 \times 1$.**
- 6 ✓ **5000 bits**. To turn bits into bytes, divide by 8. **What is $5000 \div 8$?**
- 7 ✓ **625 bytes**. So the whole chain is pixels across $\times$ pixels down $\times$ bits per pixel, then $\div 8$ for bytes. **Read on.**
- 8 Often the size is given in inches at a certain **dpi** (dots per inch). Get the pixels first: pixels = inches $\times$ dpi, the same way for width and for height. Take 2 inches at 300 dpi. **How many pixels is 2×300 ?**
- 9 ✓ **600 pixels**. Do that for the width and the height, then feed the two pixel counts into the size formula as before. **Read on.**
- 10 One last caution. A **megabyte** can mean 1024×1024 bytes or a plain one million bytes, and the two give slightly different answers, so always say which you used. That is the whole skill. Now work the example below with the answer covered.

WORKED EXAMPLE

THE METHOD, VERTICALLY

An image is 17.12 inches by 9.45 inches at 250 dpi, 1 bit per pixel. Find its memory in megabytes. **The exam sets exactly this; sit it verbatim in the final mock.**

GIVEN 17.12 in × 9.45 in · 250 dpi · 1 bit per pixel

FORMULA pixels = inches × dpi (both ways) · bytes = pixels × bits / 8 · MB = bytes / 1024 / 1024

pixels across = 17.12 × 250 = 4280

pixels down = 9.45 × 250 = 2362.5

total pixels = 4280 × 2362.5 = 10 111 500

bytes = 10 111 500 / 8 = 1 263 937.5

MB = 1 263 937.5 / 1024 / 1024 = 1.21 MB

10 111 500 pixels × 1 bit = 1 263 937.5 bytes ≈ 1.21 MB. Say which megabyte you used (1024 twice gives 1.21; a million gives about 1.26).

Now redo it: pixels first, times the bits per pixel, divide by 8, then by 1024 twice.

YOUR TURN

MCQ, IMAGES

(1) How many bits per pixel for a monochrome bitmap? (2) A logo must be enlarged to billboard size without losing quality: bitmap or vector? (3) Photographs are best represented as?

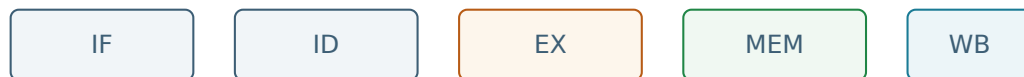
ANSWER

(1) **1 bit** (two states). (2) **vector** (scales with no quality loss). (3) **bitmap** (a photo is a grid of continuous-tone pixels).

PART B

The Machine

The heart of Question One: instructions, how operands are found, performance sums, pipelining, and memory and cache. Every skill here is a fixed method you apply to whatever numbers the paper hands you.



Instruction Sets and Formats

What a machine instruction is, the kinds there are, and the four "address" formats the paper asks you to write a program in. Plus the counting question the objective test likes.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 A machine instruction is one small command for the CPU. It always names an operation to carry out, for example ADD. The part that says *which* operation is called the **opcode**. **Name the part of an instruction that says which operation to do.**
- 2 ✓ **the opcode.** Sitting beside the opcode are zero or more **operands**: the things the operation works on. In `ADD A, B` the opcode is ADD, and A and B are the operands. (Each operand also carries an addressing mode saying where it is; that is the next section.) **In `ADD A, B`, what are A and B called?**
- 3 ✓ **the operands.** Opcode and operands are just **bit fields** packed side by side. The fixed layout of those fields is the instruction **format**. **What word means the layout of the bit fields in an instruction?**
- 4 ✓ **the format** (the instruction format). And the whole collection of instructions one CPU understands is called its **instruction set**. That is the vocabulary. Now the kinds of instructions. **Read on.**
- 5 There are four main **types**. First, **data transfer**: LOAD, STORE, MOVE. These move data between registers and memory, and a LOAD in particular reads **from** memory into the CPU. **Does LOAD read from memory, or write to it?**
- 6 ✓ **reads from memory.** Second, **arithmetic and logic**: ADD, SUB, AND, OR. This is the ALU's work. Third, **input/output**: talk to devices. **Which unit does the ADD and AND work?**
- 7 ✓ **the ALU.** Fourth, and the one the exam cares about, **control transfer**: JUMP, BRANCH, CALL. Normally instructions run one after another in order; these **change that order**. **Which type changes the order instructions run in?**
- 8 ✓ **control transfer.** Now the four **address formats**. This is only a count: how many operand addresses each instruction names. We go from most to fewest. **Read on.**
- 9 **Three-address** names both sources and the destination in one instruction, e.g. `ADD X, A, B` meaning $X = A + B$. Because each instruction does so much, you need the **fewest** of them. **How many addresses does `ADD X, A, B` name?**
- 10 ✓ **three.** **Two-address** drops the separate destination; the destination doubles as one source, e.g. `ADD A, B` means $A = A + B$. **After `ADD A, B`, where has the result gone?**
- 11 ✓ **into A.** **One-address** works through a single register called the **accumulator**: LOAD a value in, do the operation, then STORE the result out. Only one address is named each time. **Read on.**

- 12 **Zero-address** names **no** address at all. It is a **stack** machine: you PUSH operands onto the stack, and ADD simply pops the top two and pushes their sum. **Which of the four formats is the stack machine?**
- 13 ✓ **zero-address**. Now the trade-off across all four. Fewer addresses means each instruction is **shorter**, but you need **more** of them. **Which format gives the shortest program (fewest instructions)?**
- 14 ✓ **three-address**. And the other end: **zero-address** gives the longest program (most instructions), though each one is tiny. You will write one expression in all four formats in the worked example below. **Read on.**
- 15 Now the counting question the objective test likes. To size a program in bytes, count each instruction and add up, using: bytes per instruction = opcode bytes + (number of addresses × bytes per address). **Read on.**
- 16 A single instruction's total **length** is just its fields added together. Take an 8-bit opcode, a 6-bit register field and an 18-bit address field. **Add them; what is the total length in bits?**
- 17 ✓ **32 bits**. ($8 + 6 + 18 = 32$.) When every instruction is the **same** size it is called **fixed-length**; these decode faster and are used by RISC. Variable-length instructions are used by CISC. **Which style, fixed or variable, is faster to decode?**
- 18 ✓ **fixed-length**. Last one: to fit **more opcodes** into the same instruction length, you give the opcode **more bits** (and the operands fewer). That is every idea here. Now watch one expression written in all four formats in the worked example below, then sit the recall with the answer covered.

● MUST DO

THE COUNTING FORMULA

bytes per instruction = opcode bytes + (number of addresses × bytes per address)

A single instruction's total length is its fields added: an 8-bit opcode + a 6-bit register field + an 18-bit address field = **32 bits**. **Fixed-length** (all one size) decodes faster and is used by RISC; **variable-length** by CISC. More opcodes in the same length means giving the opcode more bits.

WORKED EXAMPLE**WRITE $X = (A + B) \times (C + D)$**

The same expression in each format. Watch the instruction count climb as the number of addresses drops.

Three-address (shortest, 3 instructions):

```
ADD  T, A, B      ; T = A + B
ADD  X, C, D      ; X = C + D
MUL  X, T, X      ; X = T * X
```

Two-address (destination is also a source):

```
MOV  T, A
ADD  T, B          ; T = A + B
MOV  X, C
ADD  X, D          ; X = C + D
MUL  X, T          ; X = X * T
```

One-address (one accumulator, AC):

```
LOAD  A
ADD  B              ; AC = A + B
STORE T
LOAD  C
ADD  D              ; AC = C + D
MUL  T              ; AC = AC * T
STORE X
```

Zero-address (stack, most instructions but each is tiny):

```
PUSH A   PUSH B   ADD      ; A + B on the stack
PUSH C   PUSH D   ADD      ; C + D on the stack
MUL                      ; (A+B) * (C+D)
POP  X
```

The bigger version, $X = ((A \times B) + C/D) / (F \times G + J \times K)$, is Question 4b on the 2025/2026 paper and is in your final mock at the back.

YOUR TURN**MCQ, INSTRUCTIONS**

(1) Which format is used by stack machines? (2) Which instruction category changes the program's execution sequence? (3) An instruction has an 8-bit opcode, a 6-bit register field and an 18-bit address field: total length? (4) Which is NOT defined by the ISA: addressing modes, instruction formats, internal ALU implementation, or visible registers?

ANSWER

(1) **zero-address** (operands on the stack top). (2) **control transfer** (branch/jump/call). (3) **8 + 6 + 18 = 32 bits**. (4) **internal ALU implementation** (that is organization, not the programmer-visible ISA).

⚠ TRAP

"Zero-address" goes with the **stack** and operands on **top of the stack**; do not mix it up with three-address (the shortest program). And the ISA defines what the programmer sees (formats, modes, registers), never how the ALU is wired inside.

THE MACHINE

Addressing Modes

An address field in an instruction does not always hold the operand directly. The addressing mode says how to turn that field into the real location, the effective address, and then the operand. Learn the six or seven modes as "how many hops through memory", and this is easy marks on both the paper and the objective test.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Here is the one idea behind every addressing mode. An instruction carries an address **field**, but that field does not always hold the operand you want. The **mode** is the rule for turning the field into the real place, and then the operand. **What does the addressing mode tell you how to find?**
- 2 ✓ **the real location, then the operand.** Two names to fix first. The real location of the operand is the **effective address**, or EA. And **M[x]** is shorthand for "the value stored at address x", so M[1200] is the value sitting at address 1200. **In words, what does M[4500] mean?**
- 3 ✓ **the value stored at address 4500.** We will grade every mode by one question: **how many hops through memory** does it take to reach the operand? We start with the fewest. **Read on.**
- 4 **Immediate** mode. The operand is a plain constant written **inside** the instruction itself, so you never touch memory to fetch it. **How many memory hops does immediate mode need?**
- 5 ✓ **zero.** Needing no memory access, immediate is the fastest way to supply a constant. Next, **direct** mode: the field is the address, so EA = the field and the operand = M[field], one hop into memory. **Direct mode reads memory how many times?**
- 6 ✓ **once** (one hop). **Indirect** mode adds one more hop. Now the field holds the address of the address: first EA = M[field], then the operand = M[M[field]]. That is two hops, a pointer to a pointer. **How many hops does indirect mode take?**
- 7 ✓ **two.** Because indirect follows an address that is itself stored in memory, it is exactly what you need to chase pointers, for example down a linked list. **Read on.**
- 8 Two modes lean on a **register** instead of memory. **Register** mode: the operand sits in a named register, so no memory hop at all. **Register indirect**: the register holds an **address**, so the operand is M[register], one hop. **In register-indirect, is the operand in the register, or at the address the register holds?**
- 9 ✓ **at the address the register holds.** Last group, the "add" modes; each **adds** something to the field to make the EA. **Relative**: EA = PC + field. **Indexed**: EA = index register + field. **Base**: EA = base register + displacement. **What does relative mode add to the field?**
- 10 ✓ **the PC.** Now put it to work on one running example. Take PC = 7000, R1 = 1500, address field = 1200, and memory holds M[1200] = 4500, M[4500] = 999, M[1500] = 6000. Keep these in view. First, **direct** (EA = field, operand = M[field]). **Give the direct operand.**

- 11 ✓ **4500**. (EA = 1200, operand = M[1200] = 4500.) Now **indirect** on the same field: first EA = M[1200], then operand = M[that]. **Work both hops and give the indirect operand.**
- 12 ✓ **999**. (EA = M[1200] = 4500, then operand = M[4500] = 999.) Now **register indirect** with R1 = 1500: the register holds the address, so operand = M[R1]. **Give the register-indirect operand.**
- 13 ✓ **6000**. (R1 = 1500, so operand = M[1500] = 6000.) Finally **relative**, an add-mode: EA = PC + field, with PC = 7000 and field = 1200. **Give the relative EA.**
- 14 ✓ **8200**. (7000 + 1200 = 8200. The table gives no M[8200], so you would report the EA and never invent a value.) **Read on.**
- 15 Now the whole thing on one card. Immediate = the operand is **in** the instruction (zero hops). Direct = **one** hop. Indirect = **two** hops. And the add-modes (relative, indexed, base) **add** a register or the PC to the field first, then read there. **Order these by hops, fewest first: direct, immediate, indirect.**
- 16 ✓ **immediate (0), direct (1), indirect (2)**. That is every mode you need. Now study the worked example and recall below, which run this exact PC = 7000, R1 = 1500, field = 1200 example, and sit them with the answers covered.

● MUST DO

THE MODES, AS HOPS

The **effective address (EA)** is where the operand really is; **M[x]** means "the value stored at address x". Each mode is one more hop than the last:

MODE	EFFECTIVE ADDRESS	OPERAND IS
Immediate	none	the value inside the instruction (a constant)
Direct	EA = the address field	M[address field]
Indirect	EA = M[address field]	M[M[address field]]
Register	none	the value in the named register
Register indirect	EA = contents of the register	M[register]
Relative (PC-relative)	EA = PC + address field	M[PC + field]
Indexed	EA = index register + address field	M[index + field]
Base	EA = base register + displacement	M[base + displacement]

Memory hook: immediate = **in** the instruction (zero hops, fewest memory accesses); direct = **one** hop; indirect = **two** hops (a pointer to a pointer). Register indirect uses a register as the pointer; relative, indexed and base all **add** something to the field.

WORKED EXAMPLE
THE METHOD, LAID OUT

PC = 7000, R1 = 1500, address field = 1200. Memory holds: M[1200] = 4500, M[4500] = 999, M[1500] = 6000. Give the EA and operand for each mode. **The exam sets exactly this; sit it verbatim in the final mock.**

MODE	EFFECTIVE ADDRESS	OPERAND
Immediate	none	the field itself = 1200
Direct	1200	M[1200] = 4500
Indirect	M[1200] = 4500	M[4500] = 999
Register	none	R1 = 1500
Register indirect	R1 = 1500	M[1500] = 6000
Relative	PC + 1200 = 8200	M[8200], not in the given table

Immediate 1200; Direct 4500; Indirect 999; Register 1500; Register indirect 6000; Relative EA 8200. If the table has no entry for an EA, say so, do not invent a value.

Now redo it: follow the hops. Direct reads memory once, indirect twice, the "add" modes add a register to the field first.

YOUR TURN
MCQ, WORK THE EA

(1) R1 = 2000, M[2000] = 500. Register-indirect operand? (2) Instruction address 500, displacement +30, PC-relative EA? (3) R2 = 300 (index), displacement 20, indexed EA? (4) Base register 4000, displacement 120, base EA? (5) Address 700, M[700] = 1500, M[1500] = 60. Indirect operand?

ANSWER

(1) M[2000] = **500**. (2) 500 + 30 = **530**. (3) 20 + 300 = **320**. (4) 4000 + 120 = **4120**. (5) M[700] = 1500, then M[1500] = **60**.

YOUR TURN
MCQ, PICK THE MODE

(1) A constant is embedded in the instruction: which mode? (2) Which mode needs the fewest memory accesses? (3) Which mode best suits following pointers in a linked list?

ANSWER

(1) **immediate**. (2) **immediate** (the operand is already in the instruction, no memory hop). (3) **indirect** (it dereferences a pointer, which is exactly what a linked list needs).

⚠ TRAP

When a question gives you $M[EA]$, read what it asks: the **effective address** is the location, the **operand** is the value there. For indexed, base and relative modes, you **add** the register or PC to the field to get the EA; the value stored at that EA is the operand, one further hop.

THE MACHINE

CPU Performance

How fast a processor runs is a calculation, not a mystery. Two formulas do almost everything: the CPU-time equation and Amdahl's law. This is the most heavily weighted numeric topic on the recent paper.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Everything about processor speed comes from one equation. A program's running time is three numbers multiplied together: $\text{CPU time} = \text{IC} \times \text{CPI} \times \text{T}$. IC is the instruction count (how many instructions actually run), CPI is the average cycles per instruction, and T is how long one clock cycle lasts. **Say what each of the three, IC, CPI and T, stands for, then slide down.**
- 2 ✓ **instruction count, average cycles per instruction, clock cycle time.** The clock rate you see advertised (say 4 GHz) is cycles per second, so one cycle lasts $\text{T} = 1 / \text{clock rate}$. **Put that in place of T and write CPU time using clock rate.**
- 3 ✓ **CPU time = (IC × CPI) / clock rate.** So a faster clock shortens the time, but a smaller instruction count or a smaller CPI does too. All three matter. **Read on.**
- 4 Look closer at CPI. Instructions are not all equal: a register add may take 1 cycle, an indirect memory access 4. The average CPI weights each type by how often it appears, adding up (fraction of the type × its cycles) over all types. The fractions must add to 1. **Read on.**
- 5 Try a tiny one. A program is 70% simple instructions at 1 cycle each and 30% at 3 cycles each. **Work out the average CPI: $(0.70 \times 1) + (0.30 \times 3)$.**
- 6 ✓ **1.60.** ($0.70 \times 1 = 0.70$, and $0.30 \times 3 = 0.90$, giving 1.60.) Notice 0.70 and 0.30 add to 1, exactly as the fractions must. **Read on.**
- 7 A second speed measure is MIPS, millions of instructions per second: $\text{MIPS} = \text{clock rate} / (\text{CPI} \times 10^6)$. A faster clock or a lower CPI both raise it. **If the clock rate is 2×10^9 and CPI is 2, find the MIPS.**
- 8 ✓ **1000 MIPS.** (2×10^9 divided by (2×10^6) gives 1000.) **Read on.**
- 9 When you improve something, always measure the gain against the OLD time: $\text{improvement} = (\text{old} - \text{new}) / \text{old}$. **Old time 200, new time 150: what fraction is the improvement?**
- 10 ✓ **0.25, that is 25%.** ($(200 - 150) / 200 = 50 / 200 = 0.25$.) Divide by the old value, never the new. **Read on.**
- 11 So is a chip fast just because its clock is fast? No. Because $\text{time} = (\text{IC} \times \text{CPI}) / \text{clock rate}$, the instruction count and CPI count just as much, and behind CPI sit cache behaviour, branch prediction and instruction-level parallelism. **Name one reason a high-clock chip can still be slow.**
- 12 ✓ **a high CPI** (or a large instruction count, or poor cache or branch behaviour). That is why "judge a processor by clock frequency only" is wrong. **Read on.**

- 13 Now the second big formula, Amdahl's law. Suppose you speed up only PART of a task. The part you did not touch still takes its full time, so it caps the gain. Let f be the fraction of the original time spent in the part you improve, and let s be how many times faster you make that part. **In your own words, what does f stand for?**
- 14 ✓ **the fraction of the time spent in the part you speed up.** That part's time shrinks from f to f / s , while the rest, $(1 - f)$, is unchanged. If the whole job was time 1, the new time is $(1 - f) + f / s$, and speedup is old over new. **Write the speedup S .**
- 15 ✓ **$S = 1 / ((1 - f) + f / s)$.** Old time 1 on top, new time $(1 - f) + f / s$ on the bottom. **Read on.**
- 16 Push s as large as you like, so f / s shrinks towards 0. Then S climbs only as far as $1 / (1 - f)$, because the untouched $(1 - f)$ never goes away. Memory hook: the slow part you left alone sets the ceiling. **With $f = 0.85$, what is the best speedup possible, even with an infinite s ?**
- 17 ✓ **about 6.67.** ($1 / (1 - 0.85) = 1 / 0.15 = 6.67$.) Now a real, finite case: a cache 10 times faster ($s = 10$) that the CPU hits 85% of the time ($f = 0.85$). First find the new time, $(1 - f) + f / s$. **Compute $0.15 + 0.85 / 10$.**
- 18 ✓ **0.235.** ($0.15 + 0.085 = 0.235$.) So $S = 1 / 0.235$. **Finish it: what is S ?**
- 19 ✓ **about 4.26.** ($1 / 0.235 = 4.26$.) Far below the cache's 10 times, because the 15% of slow accesses dominate, which is exactly Amdahl's point. The reference card and the two worked examples below sit both of these formulas; do them with the answers covered.

● MUST DO

THE PERFORMANCE FORMULAS ON ONE CARD

- ▶ **CPU time** = $IC \times CPI \times T = (IC \times CPI) / \text{clock rate}$ (since $T = 1 / \text{clock rate}$).
- ▶ **Average CPI** = $\Sigma(\text{fraction of each type} \times \text{its cycles})$, and the fractions add to 1.
- ▶ **MIPS** = $\text{clock rate} / (CPI \times 10^6)$.
- ▶ **Improvement** = $(\text{old} - \text{new}) / \text{old}$, always against the old time.
- ▶ **Amdahl's law:** $S = 1 / ((1 - f) + f / s)$, ceiling $1 / (1 - f)$.

Memory hook: the slow part you left alone sets the ceiling, no matter how fast you make the rest.

WORKED EXAMPLE
THE METHOD, VERTICALLY

One million instructions, by addressing mode: Register 40% at 1 cycle, Immediate 20% at 1, Direct 15% at 2, Indirect 10% at 4, Indexed 10% at 3, Relative 5% at 2. Find the average CPI and total cycles; then optimise indirect to 2 cycles and give the improvement. **The exam sets exactly this; sit it verbatim in the final mock.**

GIVEN IC = 1 000 000 · the share and cycle cost of each mode

FORMULA $CPI = \Sigma(\text{fraction} \times \text{cycles}) \cdot \text{total} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new}) / \text{old}$

$$\begin{aligned} \text{Avg CPI} &= 0.40 \times 1 + 0.20 \times 1 + 0.15 \times 2 + 0.10 \times 4 + 0.10 \times 3 + 0.05 \times 2 \\ &= 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10 \\ &= 1.70 \end{aligned}$$

$$\text{Total cycles} = 1\,000\,000 \times 1.70 = 1\,700\,000$$

$$\begin{aligned} \text{Indirect 4} \rightarrow 2: \text{ new CPI} &= 1.70 - 0.10 \times 4 + 0.10 \times 2 \\ &= 1.70 - 0.20 \\ &= 1.50 \end{aligned}$$

$$\text{Improvement} = (1.70 - 1.50) / 1.70 = 0.20 / 1.70 = 11.76\%$$

CPI 1.70; total cycles 1 700 000; new CPI 1.50; improvement 11.76%.

Now redo it: weighted average, then recompute with the one changed cost.

WORKED EXAMPLE
THE METHOD, VERTICALLY

A cache is 10 times faster than main memory, and the CPU finds what it needs in cache 85% of the time. Find the overall speedup. **The exam sets exactly this; sit it verbatim in the final mock.**

GIVEN $f = 0.85$ (cache hits) · $s = 10$

FORMULA $S = 1 / ((1 - f) + f/s)$

$$\begin{aligned} S &= 1 / ((1 - 0.85) + 0.85/10) \\ &= 1 / (0.15 + 0.085) \\ &= 1 / 0.235 \\ &= 4.26 \end{aligned}$$

$S = 4.26$, far below 10, because the 15% of slow accesses dominate. That is exactly Amdahl's point.

Now redo it: the "10 times faster" is s , not the answer; only the 85% gets divided by s , the other 15% stays at full cost.

(1) A benchmark runs 12 billion instructions at 4 GHz with an average CPI of 1.5. Execution time? (2) Why should performance not be judged by clock frequency alone?

ANSWER

(1) $\text{time} = (\text{IC} \times \text{CPI}) / \text{clock rate} = (12 \times 10^9 \times 1.5) / (4 \times 10^9) = 18 / 4 = \mathbf{4.5 \text{ seconds.}}$
(2) because it also depends on **CPI, instruction count, cache behaviour and instruction-level parallelism**, not clock speed by itself.

⚠ TRAP

Improvement is measured against the **old** time, $(\text{old} - \text{new})/\text{old}$, not the new. In Amdahl's law, **f** is the fraction sped up and only that part is divided by **s**; a "10 times faster" cache with 85% hits gives about 4.26, not 10.

THE MACHINE

Pipelining and Hazards

Overlapping instructions so several are in progress at once, like a laundry that washes one load while drying the last. It speeds the processor up, and the paper asks you to compute that speedup and to name the hazards that get in the way.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Pipelining means overlapping instructions so several are in progress at once. Picture a laundry: you do not wait for one load to wash, dry and fold before starting the next, because the moment the first load moves to the dryer the second goes into the washer. A processor overlaps its instructions the same way. **What everyday process is pipelining most like?**
- 2 ✓ **a laundry** (or any assembly line). To overlap them, we split each instruction into stages, called k stages. A common five are IF (instruction fetch), ID (instruction decode), EX (execute), MEM (memory access) and WB (write back). Each stage is its own hardware, so five instructions can each sit in a different stage at the same moment. **Name the five stages in order.**
- 3 ✓ **IF, ID, EX, MEM, WB.** Without a pipeline, instruction 2 cannot start until instruction 1 has passed through every stage. With a pipeline, the instant an instruction leaves a stage the next one enters it, so once the pipe is full one instruction finishes every clock cycle. **Once the pipe is full, how many instructions complete per clock cycle?**
- 4 ✓ **one every cycle.** That steady one-per-cycle output is the whole payoff. Now, how long is one clock cycle? Every stage must finish inside it, so the cycle is set by the slowest stage plus a small latch delay between stages, never the average and never the sum. **If the stages take 60, 50, 90, 80 ns with a 10 ns latch, what is the cycle time?**
- 5 ✓ **100 ns.** (The slowest stage is 90 ns, plus the 10 ns latch, giving 100 ns.) **Read on.**
- 6 Now the time for n instructions. The first takes k cycles to fill the pipe; after that one more finishes every cycle, so n instructions take $(k + n - 1)$ cycles, which you then multiply by the cycle time. **With $k = 4$ stages and $n = 500$ tasks, how many cycles is that?**
- 7 ✓ **503 cycles.** ($4 + 500 - 1 = 503$, and at 100 ns each that is 50 300 ns.) **Read on.**
- 8 Without the pipeline each instruction needs its full one-instruction time, the sum of the stage times, here $60 + 50 + 90 + 80 = 280$ ns. The speedup is that one-instruction time divided by the cycle time, and it can never exceed k , the number of stages. **Divide 280 ns by the 100 ns cycle to get the speedup.**
- 9 ✓ **2.8.** ($280 / 100 = 2.8$, below the ceiling of $k = 4$ stages.) For a whole batch you compare totals instead: $500 \times 280 = 140\,000$ ns without a pipeline, against 50 300 ns with one. The reference card and the two worked examples below sit exactly these; do them with the answers covered.

- ▶ **Cycle time** = slowest stage + latch delay.
- ▶ **Pipelined time for n tasks** = $(k + n - 1) \times \text{cycle time}$. (It takes k cycles to fill with the first task, then one more cycle for each of the other $n - 1$.)
- ▶ **Non-pipelined time for n tasks** = $n \times (\text{time for one instruction})$. One instruction takes the sum of all the stage times, or the given per-instruction time.
- ▶ **Speedup** = non-pipelined time / pipelined time. Its ceiling is k.

Memory hook: " $k + n - 1$ ", and the clock runs at the **slowest** stage, never the average.

WORKED EXAMPLE

THE METHOD, VERTICALLY

A pipeline has 4 phases of 60, 50, 90, 80 ns, with a 10 ns latch delay. Find the cycle time, the one-task time, the speedup, and the times for 500 tasks. **The exam sets exactly this; sit it verbatim in the final mock.**

GIVEN 4 phases 60, 50, 90, 80 ns · latch 10 ns · 500 tasks
 FORMULA cycle = slowest + latch · pipe(n) = $(k + n - 1) \times \text{cycle}$ · seq(n) = $n \times \Sigma \text{phases}$ ·
 speedup = $\Sigma \text{phases} / \text{cycle}$

Cycle = $\max(60, 50, 90, 80) + 10 = 90 + 10 = 100 \text{ ns}$
 One task = $60 + 50 + 90 + 80 = 280 \text{ ns}$
 Speedup = $280 / 100 = 2.8$
 Pipe 500 = $(4 + 500 - 1) \times 100 = 503 \times 100 = 50\,300 \text{ ns}$
 Seq 500 = $500 \times 280 = 140\,000 \text{ ns}$

Cycle 100 ns; one task 280 ns; speedup 2.8; 500 tasks pipelined 50 300 ns against sequential 140 000 ns.

A non-pipelined processor takes 60 ns per instruction. Redesign as a 6-stage pipeline with a 12 ns cycle time, running 50 instructions. Find both times and the speedup. (This is the one question on the revision deck that is not on any past paper, so learn it fresh.)

GIVEN non-pipelined 60 ns/instruction · $k = 6$ stages · cycle 12 ns · $n = 50$
FORMULA non-pipe = $n \times \text{time/instruction}$ · pipe = $(k + n - 1) \times \text{cycle}$ · speedup = non-pipe / pipe

Without pipelining = $50 \times 60 = 3000$ ns
 With pipelining = $(6 + 50 - 1) \times 12$
 = 55×12
 = 660 ns (fill 6x12=72, then 49x12=588)
 Speedup = $3000 / 660 = 4.55$

Without pipelining 3000 ns; with pipelining 660 ns; speedup 4.55.

Now redo it: $n \times \text{per-instruction time without}$, $(k + n - 1) \times \text{cycle with}$, then divide.

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Overlapping is not free: sometimes two instructions in the pipe clash, and a clash is called a hazard. There are three kinds, and you must be able to name them. The first is the structural hazard, where two instructions need the same piece of hardware in the same cycle (one memory wanted for a fetch and a data access at once). **How do you remove a structural hazard?**
- 2 ✓ **duplicate the resource** (for example, separate instruction and data memories, so each instruction has its own). The second kind is the data hazard: an instruction needs a result that an earlier instruction has not written yet. We name its three types by the order in which two instructions touch the same register. **Read on.**
- 3 RAW (read after write) is the true dependency: the second instruction reads a register the first has not written yet. It is eased by forwarding, which feeds the result straight from where it is produced to where it is needed, before the write-back stage. **Which of the three data hazards is the real dependency?**
- 4 ✓ **RAW.** The other two, WAR (write after read) and WAW (write after write), are only naming clashes over the same register, not true data flow. **Read on.**
- 5 When forwarding cannot deliver the value in time, the pipeline must stall: it inserts a bubble, an idle cycle, until the value is ready. **What is the idle cycle a stall inserts called?**
- 6 ✓ **a bubble.** The third kind is the control hazard: a branch changes which instruction runs next, but the pipe has already fetched the ones right after the branch. It is eased by branch prediction and speculative execution, whose whole purpose is to reduce control-hazard stalls. **Read on.**
- 7 Now classify one pair. The first instruction writes R3, and the second instruction reads R3. The shared register is R3, and the order is write then read. **Which data hazard is that?**
- 8 ✓ **RAW** (write then read is exactly read-after-write). That is every hazard idea in place. The box just below shows how to draw the pipeline timing diagram the paper sometimes asks for; after it, sit the two "your turn" boxes with the answers covered.

TEACH

HOW TO DRAW THE TIMING (CYCLE) DIAGRAM

The paper sometimes says "draw the pipeline timing diagram" or "draw the cycle diagram". It is always the same grid, and you draw it the same way every time:

- ▶ **Cycles across the top** (1, 2, 3, ...), **one instruction per row** down the side.
- ▶ In each cell write the **stage** that instruction is in that cycle: IF, ID, EX, (MEM), WB. Some papers add an **operand fetch (OF)** stage between ID and EX; use whatever stages the question names.
- ▶ Each instruction begins its IF **one cycle after** the instruction above it.
- ▶ A **multi-cycle execute** (say a 3-cycle multiply or a 6-cycle divide) writes EX in **that many cells** in a row, and everything behind it slides right.
- ▶ A **stall** (waiting for a hazard to clear, or for a busy stage) is a gap you mark with a bubble, written --.

The **total number of cycles** is simply the column where the **last** instruction reaches WB. That is your answer to "how many cycles".

How to remember it: "cycles across, instructions down, stages in the boxes, count to the last WB." A long EX **pushes**; a stall is a --. (You will draw exactly this for Question Two and Question Three in the final mock.)

YOUR TURN

CLASSIFY THE HAZARDS

Find the one register on both lines and read its order. (i) $R3 \leftarrow R1 \text{ op } R2$, then $R5 \leftarrow R3 \text{ op } R4$. (ii) $R3 \leftarrow R1 \text{ op } R2$, then $R1 \leftarrow R4 \text{ op } R5$. (iii) $R3 \leftarrow R1 \text{ op } R2$, then $R3 \leftarrow R6 \text{ op } R7$.

ANSWER

(i) $R3$ is written then read, so **RAW**. (ii) $R1$ is read then written, so **WAR**. (iii) $R3$ is written then written, so **WAW**. Only RAW is a real data dependency; the other two are naming clashes.

YOUR TURN

MCQ, PIPELINING

(1) The main objective of branch prediction and speculative execution is to? (2) A read-after-write clash is eased by which technique?

ANSWER

(1) **reduce pipeline stalls caused by control hazards** (keep the pipe full across branches). (2) **forwarding** (feed the result straight to where it is needed).

⚠ TRAP

The clock runs at the **slowest** stage plus latch, not the average or the sum. For n tasks the count is **$k + n - 1$** , so 500 tasks in a 4-stage pipe is 503 cycles, not 504. Classify data hazards by the shared **register** and the order it is written and read, never by the instruction name.

Memory and Cache

*The memory hierarchy carries the largest single block of objective questions, and cache mapping is the most examined calculation on the whole course. It looks fiddly, so we do it as a **programme**: many tiny steps, and you check each one before moving on. By the end the cache split will be the same four moves every time.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 Memory is arranged in **levels**, fastest to slowest: **registers**, then **cache**, then **main memory**, then **secondary storage** (disk or SSD). **Of those four, which is the fastest?**
- 2 ✓ **registers**. Picture it as your workspace: the few things in your hands (registers), the book open on your desk (cache), the shelf beside you (main memory), and the library across campus (disk). You keep what you use most closest, because reaching further costs time. **As you climb toward registers, does memory get cheaper or dearer per bit?**
- 3 ✓ **dearer**. Climbing up, memory gets faster, smaller, and more expensive per bit. The reason there are levels at all is a trade-off: you cannot have fast, big and cheap at once. **So which level has the highest cost per bit?**
- 4 ✓ **registers**. Now the two kinds of RAM. **SRAM** (static RAM) is built from **flip-flops**: fast, needs **no refresh**, but costly and low density. **Given it is fast but costly, which level is SRAM used for?**
- 5 ✓ **cache**. **DRAM** (dynamic RAM) is built from **capacitors**: dense and cheap, but slower and it **needs periodic refresh**, so it is used for **main memory**. (ROM and Flash are non-volatile; RAM, cache and DRAM are volatile, lost on power off.) **Which kind of RAM has to be refreshed?**
- 6 ✓ **DRAM**. Two facts the MCQs love: the **MAR** (memory address register) holds the address being accessed; and **access time** is how long one access takes, while **cycle time** is the minimum gap between the **starts** of two accesses. **A device with 60 ns access and 90 ns cycle can start a new access how often?**
- 7 ✓ **every 90 ns** (the cycle time, not the access time). Now, why does a small nearby cache help at all? Because programs are predictable, a habit called **locality of reference**. It comes in two kinds. **Read on.**
- 8 **Temporal** locality: a location used now is likely used again **soon** (think of a loop counter, read over and over). **Reusing the same location again soon is which locality?**
- 9 ✓ **temporal**. **Spatial** locality: once a location is used, **nearby** ones are likely used next (stepping through an array). That is exactly why the cache copies a whole **block** at a time, not one lonely byte. **Read on.**
- 10 Two limits to know by name. The **memory wall** is the widening gap between fast CPUs and slow memory, which is why a processor can still wait on data despite a big cache. **Virtual memory** lets a program use an address space **larger than the physical RAM**, with a memory management unit (MMU) translating the addresses. **Read on.**

- 11 Now the cache sum itself, gently. Picture the cache as a wall of numbered **hooks** (the **lines**), each big enough to hold one **block**, a short run of consecutive bytes copied from memory together. Far more blocks exist in memory than there are hooks, so many blocks compete for the same hook. **Read on.**
-
- 12 When the CPU names an address, that address answers **three questions**, and that is exactly why it splits into three fields. The first, from the **low** bits, is the **offset**. **Which one thing does the offset pick out: which byte in the block, which hook, or which block?**
-
- 13 ✓ **which byte inside the block.** The middle bits are the **index** (or **set**): which **hook** the block hangs on. The high bits are the **tag**: **which** of the many blocks sharing that hook is actually here, stored beside the block and compared on every access. **A tag that matches means what?**
-
- 14 ✓ **a hit** (no match is a miss). Now the sizes, in order. **Address bits** = $\log_2(\text{main memory size})$. **Offset** = $\log_2(\text{block size})$. **A 16-byte block needs how many offset bits?**
-
- 15 ✓ **4** ($\log_2 16 = 4$). **Number of lines** = cache size / block size. **Index bits** (direct mapped) = $\log_2(\text{lines})$. **A direct-mapped cache with 64 lines needs how many index bits?**
-
- 16 ✓ **6** ($\log_2 64 = 6$). For a **k-way set associative** cache, **sets** = lines / k and **set bits** = $\log_2(\text{sets})$; **fully associative** has no index at all (any block on any hook). Whatever is left over is the **tag**: **tag bits** = address bits – index (or set) – offset. The **tag directory** = lines × tag bits. **Read on.**
-
- 17 Last, the **average memory access time (AMAT)**, with hit ratio h , cache time T_c and memory time T_m . Hierarchical (cache first): $\text{AMAT} = T_c + (1 - h) \times T_m$. Simultaneous (both at once): $\text{AMAT} = h \times T_c + (1 - h) \times T_m$. **Read on.**
-
- 18 ✓ **Done.** That is the whole method: address bits from the memory, offset from the block, lines from cache over block, tag is whatever is left, then the tag directory and AMAT. Now watch it on real numbers in the three worked examples below, then drill the six extra caches with a card over each row.

● MUST DO

THE CACHE SPLIT ON ONE CARD

- ▶ **Address bits** = $\log_2(\text{main memory size})$.
- ▶ **Offset** = $\log_2(\text{block size})$. It picks the byte inside a block.
- ▶ **Number of lines** = cache size / block size.
- ▶ **Index bits** (direct) = $\log_2(\text{lines})$. For a **k-way set associative** cache, **sets** = lines / k and **set bits** = $\log_2(\text{sets})$. **Fully associative** has no index at all.
- ▶ **Tag bits** = address bits – index (or set) bits – offset bits. The **tag directory** is lines × tag bits.

Hierarchical (cache first): $AMAT = T_c + (1 - h) \times T_m$. Simultaneous (both at once): $AMAT = h \times T_c + (1 - h) \times T_m$.

WORKED EXAMPLE

THE CACHE SLIDES' STANDARD CASE

Main memory 64 KB, cache 1 KB, block 16 B, direct mapped. Find the address split and the tag-directory size.

```
Address = log2(64 KB) = log2(216)      = 16 bits
Offset  = log2(16)                = 4 bits
Lines   = cache/block = 1024/16 = 64
Index   = log2(64)                = 6 bits
Tag      = 16 - 6 - 4              = 6 bits
Tag dir  = lines × tag = 64 × 6 = 384 bits = 48 bytes
```

Offset 4, index 6, tag 6; tag directory 384 bits (48 bytes).

Now redo it: address from the memory size, offset from the block, lines from cache over block, and the tag is whatever is left. This works for any numbers.

WORKED EXAMPLE

AVERAGE ACCESS TIME

Hit rate 80%, cache time 5 ns, memory time 100 ns. Find the AMAT both ways.

```
Simultaneous = h.Tc + (1-h).Tm = 0.8×5 + 0.2×100 = 4 + 20 = 24 ns
Hierarchical = Tc + (1-h).Tm = 5 + 0.2×100 = 5 + 20 = 25 ns
```

Simultaneous 24 ns, hierarchical 25 ns. Hierarchical pays the cache time on every access, so it is a touch slower here.

WORKED EXAMPLE
THE REVISION DECK'S SIX EXTRA CACHES

The lecturer's revision deck sets six more cache problems (Examples 2 and 3 of each mapping). They are not on the 2025/2026 paper, so drill the method on these fresh numbers. Cover a row, rebuild it from the four sizes, then check.

CACHE (MEMORY / CACHE / BLOCK)	OFFSET	INDEX/SET	TAG	LINES	TAG DIRECTORY
Direct, 128 KB / 4 KB / 32 B	5	index 7	5	128	640 bits (80 bytes)
Direct, 1 MB / 8 KB / 64 B	6	index 7	7	128	896 bits (112 bytes)
Fully assoc, 256 KB / 4 KB / 32 B	5	none	13	128	1664 bits (208 bytes)
Fully assoc, 2 MB / 16 KB / 64 B	6	none	15	256	3840 bits (480 bytes)
4-way, 512 KB / 8 KB / 32 B	5	set 6	8	256	2048 bits (256 bytes)
8-way, 4 MB / 32 KB / 64 B	6	set 6	10	512	5120 bits (640 bytes)

The lines are set by cache over block; the tag directory grows as the mapping gets more associative, because dropping the index lengthens the tag on every line.

YOUR TURN
MCQ, MEMORY

(1) Order fastest to slowest: registers, cache, main memory, secondary. (2) SRAM versus DRAM in one line? (3) Reusing the same location soon is which locality? (4) A program using more address space than physical RAM uses what? (5) Which memory has the highest cost per bit?

ANSWER

(1) **registers, cache, main memory, secondary.** (2) SRAM = flip-flops, fast, no refresh; **DRAM = capacitors**, dense, needs refresh. (3) **temporal** (nearby locations would be spatial). (4) **virtual memory.** (5) **registers.**

⚠ TRAP

Keep the units straight: offset comes from the **block** size, lines from **cache over block**. For set associative, divide lines by the ways to get **sets** before taking the log, and the bits you save on the index move into the **tag**. Temporal locality is "same place, later"; spatial is "nearby places".

PART C

Prove It

The last theory, then the test. Learn RISC versus CISC and a handful of loose ideas, then work the whole MCQ bank and sit the 2025/2026 paper under the clock.

RISC

vs

CISC

RISC, CISC and the Rest

*Two design philosophies, plus the loose theory ideas the objective test dips into. Mostly describing and comparing, so we do it as a short **programme**: one small contrast at a time, then check. Get the contrasts clean and these are fast, reliable marks.*

Cover the page just below the first line with a card or your hand. Read a frame, do the small task, then slide the card down to the next line to check your answer. One step at a time, and do not skip.

- 1 An instruction set can be designed with two opposite philosophies. The first is **CISC** (complex instruction set computer): **many**, powerful, **variable-length** instructions, some doing a lot of work each (even direct memory-to-memory operations). **Does CISC have few or many instructions?**
- 2 ✓ **many**. Because each instruction does so much, CISC programs are **short**, but the hardware is complex, instructions take several cycles, and it is **harder to pipeline**. Its instructions can resemble high-level language constructs, which **eases the compiler**. It suited the old days of slow, expensive memory. **Read on.**
- 3 The opposite is **RISC** (reduced instruction set computer): **few**, simple, **fixed-length** instructions, each ideally **one cycle**. **Are RISC instructions fixed-length or variable-length?**
- 4 ✓ **fixed-length**. RISC touches memory only through **load and store** (a load-store design), keeps **many registers**, and uses hardwired control. Programs are longer, but the hardware is simple, fast, and **pipelines cleanly**. It minimises chip area and power and leans on the **compiler**. **So which philosophy pipelines more easily, RISC or CISC?**
- 5 ✓ **RISC** (simple fixed-length instructions flow through the pipeline cleanly). One quick sort for the exam: **variable-length instruction formats go with RISC or CISC?**
- 6 ✓ **CISC** (RISC is fixed-length). The card just below lays the two side by side. Examples to recognise: RISC is ARM, MIPS, SPARC, PowerPC; CISC is Intel x86, VAX, the Motorola 68000. **Read on to the loose ideas the MCQs also test.**
- 7 **SIMD** (single instruction, multiple data): one instruction acts on **many data items at once**, to speed up data-parallel work like graphics and matrices. **Why does SIMD exist?**
- 8 ✓ **to accelerate data-parallel computations**. **Domain-specific architecture**: a processor tailored to one job (say the matrix work in AI) can be far more efficient than a general-purpose one for that job. **Read on.**
- 9 **Abstraction**: layering the design, with the ISA as the key layer, lets the hardware and software teams work **independently** while staying compatible. **Read on.**
- 10 **Backward compatibility**: keeping a **stable ISA** while the organization underneath evolves is what lets twenty-year-old software still run on new chips. **What one thing gives backward compatibility across processor generations?**

11 ✓ a **stable ISA** (with the organization evolving underneath). Last one, a wording the test likes: **architecture versus engineering**. Computer architecture is the conceptual, operational design; computer engineering is the broader design, build and integration of the hardware. **Read on.**

12 ✓ **Done.** You have now met every skill the exam tests. What is left is **proof**: turn the page and work the MCQ bank a whole topic at a time with the answers covered, going back to any skill you miss, then sit the 2025/2026 paper at the very back under exam conditions.

● MUST DO

RISC VERSUS CISC, SIDE BY SIDE

RISC	CISC
Few, simple, fixed-length instructions	Many, complex, variable-length
Mostly one cycle each; pipelines well	Several cycles; harder to pipeline
Load-store; many registers	Memory-to-memory; fewer registers
Complexity handled by the compiler	Complexity handled by the hardware

YOUR TURN

MCQ, RISC/CISC AND FRIENDS

(1) Variable-length instructions go with RISC or CISC? (2) A chip that minimises area and power and leans on the compiler follows which philosophy? (3) Why does SIMD exist? (4) What gives backward compatibility across processor generations?

ANSWER

(1) **CISC** (RISC is fixed-length). (2) **RISC**. (3) to **accelerate data-parallel computations**. (4) a **stable ISA** while the organization evolves underneath.

● MUST DO

BEFORE YOU TURN THE PAGE

You have now met every skill the exam tests. What is left is **proof**. Turn the page and work the **MCQ bank**: attempt a whole topic with the answers covered, then check. Anything you miss, go straight back to the skill that teaches it. Then sit the **2025/2026 paper** at the very back under exam conditions. That is the whole plan.

PROVE IT

The MCQ Bank, Every Question Answered

Every objective question from both practice tests, grouped by topic and answered. Cover the answers, attempt a whole topic, then check. If you miss one, go back to the skill that teaches it.

Architecture versus Organization

1. An engineer describes computer architecture as the "blueprint" of a system. Which analogy best represents computer organization? (a) The owner of the building (b) The cost estimate (c) The construction process implementing the design (d) The building design
2. Which of the following is NOT an example of a computer architecture component? (a) Memory technology used (b) Addressing modes (c) Register organization (d) Instruction set
3. A cloud computing provider improves application response time by introducing multiple levels of cache memory. This improvement is mainly related to: (a) Data representation (b) Character encoding (c) ISA design (d) Memory organization
4. A laptop and a desktop both use x86 processors. The desktop performs significantly better due to more CPU cores and larger caches. This performance difference arises mainly from: (a) Organizational differences (b) Different programming languages (c) Different operating systems (d) Architectural incompatibility
5. A bank upgrades its servers to processors with larger caches and faster RAM while retaining the same x86 instruction set. What has primarily changed? (a) Architecture only (b) Organization only (c) Both architecture and organization (d) Neither architecture nor organization
6. A computer scientist states that architecture and organization must be studied together because: (a) Organization alone determines functionality. (b) Architecture alone determines performance. (c) Both concepts refer to the same thing. (d) Architecture defines capabilities, while organization determines efficiency and performance.
7. A national supercomputing center is evaluating two competing processor designs. Processor X achieves higher benchmark performance but consumes significantly more power. Processor Y delivers slightly lower performance while offering substantially better energy efficiency and scalability. Considering contemporary computer architecture principles for high-performance computing, which evaluation is the most appropriate? (a) Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone. (b) Both processors are architecturally equivalent. (c) Processor X is always preferable because peak performance is the sole design objective. (d) Energy efficiency has no relevance in processor architecture.
8. A research laboratory designs a custom processor specifically optimized for matrix multiplication in artificial intelligence workloads. This design illustrates which architectural principle? (a) General-purpose optimization always outperforms specialization. (b) Matrix operations require no architectural optimization. (c) Specialized architectures eliminate software dependence. (d) Domain-specific architectures can significantly improve efficiency for targeted applications.
9. Which statement best characterizes the distinction between computer architecture and computer engineering? (a) Computer engineering excludes hardware design. (b) Both disciplines are identical. (c) Computer architecture concerns only software. (d) Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems.

10. Two processors implement exactly the same Instruction Set Architecture but exhibit significantly different execution times when running identical benchmark applications. Which explanation best accounts for this observation? (a) Different ISAs always produce identical performance. (b) The compiler determines all performance differences regardless of hardware. (c) Identical ISAs require identical transistor layouts. (d) Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.
11. Which statement most accurately reflects the role of abstraction in computer architecture? (a) Abstraction eliminates hardware complexity entirely. (b) Abstraction reduces processor speed. (c) Abstraction only applies to programming languages. (d) Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility.
12. A processor supports backward compatibility with software written twenty years ago despite significant improvements in hardware implementation. Which architectural philosophy is primarily responsible for this capability? (a) Eliminating compiler optimizations. (b) Maintaining a stable ISA while evolving the underlying organization and microarchitecture. (c) Replacing the ISA every processor generation. (d) Increasing clock frequency only.

ANSWERS

ANSWERS

1. **c** architecture is the blueprint, organization is the construction that implements it.
2. **a** memory technology is an organization detail, not architecture. 3. **d** adding cache levels is a memory organization choice. 4. **a** same ISA, so the gap is organizational.
5. **b** the ISA is unchanged, only the organization improved. 6. **d** architecture sets capabilities, organization sets efficiency and performance. 7. **a** modern evaluation weighs power, thermal limits, scalability and cost, not peak speed alone. 8. **d** domain-specific architectures improve efficiency for targeted workloads. 9. **d** architecture is conceptual and operational design, engineering is the broader hardware build. 10. **d** clock, cache, execution units and pipeline vary despite an identical ISA. 11. **d** abstraction separates concerns across layers so hardware and software teams work independently. 12. **b** a stable ISA with an evolving organization preserves backward compatibility.

ISA and Microarchitecture

1. A game console experiences improved frame rates because the compiler uses efficient memory-access techniques. Which architectural component is most directly involved? (a) Control signals (b) System buses (c) Addressing modes (d) Memory chips
2. Which of the following would most likely be visible to a programmer? (a) Memory chip technology (b) Cache implementation details (c) Control signal generation circuitry (d) Instruction formats
3. A programmer wants to know whether a processor supports floating-point instructions. This concern relates to: (a) Control signals (b) Memory hierarchy (c) Bus organization (d) Computer architecture
4. A compiler attempts to minimize memory accesses by keeping frequently used variables inside processor registers. Which instruction category benefits the most? (a)

- Register-register arithmetic (b) Privileged instructions (c) Input/output instructions (d) Interrupt instructions
5. Which statement best distinguishes ISA from microarchitecture? (a) ISA specifies transistor layout. (b) ISA determines cache replacement policies. (c) ISA specifies the programmer-visible interface. (d) ISA defines how instructions are executed internally.
 6. Which instruction is typically executed immediately after a successful comparison operation? (a) Divide (b) Conditional Branch (c) STORE (d) LOAD
 7. Which instruction category is responsible for moving data between CPU registers and memory? (a) Floating-point instructions (b) Data transfer instructions (c) Control instructions (d) Logical instructions
 8. Which instruction would most likely access the main memory? (a) XOR (b) LOAD (c) SHIFT (d) ADD
 9. Why are specialized instructions such as SIMD instructions introduced into modern processors? (a) To increase instruction fetch time (b) To simplify cache design (c) To eliminate registers (d) To accelerate data-parallel computations
 10. A software developer successfully runs the same compiled application on two processors from different manufacturers. This is possible because both processors implement the same: (a) Instruction Set Architecture (b) Microarchitecture (c) Control unit design (d) Cache hierarchy
 11. Which instruction category primarily changes the sequence of program execution? (a) Arithmetic (b) Control Transfer (c) Data Transfer (d) Logical
 12. Which of the following is NOT defined by an Instruction Set Architecture? (a) Internal ALU implementation (b) Addressing modes (c) Instruction formats (d) Programmer-visible registers
 13. A processor supports arithmetic, logical, branch, load/store, and input/output instructions. These collectively constitute its: (a) Instruction set (b) Address bus (c) Register file (d) Pipeline stages

ANSWERS

ANSWERS

1. **c** addressing modes give the compiler efficient memory access. 2. **d** instruction formats are part of the programmer-visible ISA. 3. **d** supported instructions are defined by the architecture, the ISA. 4. **a** register-register arithmetic works straight from registers. 5. **c** the ISA is the programmer-visible interface. 6. **b** a conditional branch acts on the compare result. 7. **b** data transfer instructions move data between registers and memory. 8. **b** LOAD reads an operand from main memory. 9. **d** SIMD accelerates data-parallel computations. 10. **a** a shared Instruction Set Architecture runs the same binary. 11. **b** control transfer instructions redirect program flow. 12. **a** the internal ALU implementation is microarchitecture, not ISA. 13. **a** together they form the instruction set.

Number Systems and Codes

1. BCD stands for: (a) Binary Control Digit (b) Base Conversion Decimal (c) Binary Coded Decimal (d) Binary Character Decimal
2. The decimal value 255 represented in base 16 is: (a) FE (b) FF (c) AE (d) F0

3. Why is Excess-3 considered self-complementing? (a) Its 1's complement represents the 9's complement of the original digit (b) It eliminates overflow (c) It uses 3-bit encoding (d) It automatically detects errors
4. Why is hexadecimal preferred for displaying binary information? (a) It increases precision (b) It groups bits conveniently into sets of four (c) It reduces storage requirements (d) It speeds processor execution
5. Unicode was introduced primarily to: (a) Replace binary arithmetic (b) Support multiple world languages (c) Improve floating-point performance (d) Reduce storage
6. A company logo needs to be enlarged to billboard size without losing quality. Which format is most appropriate? (a) ASCII (b) Bitmap (c) Vector (d) BCD
7. If a hexadecimal value is shifted right by one bit, the operation is equivalent to: (a) Multiplication by 2 (b) Division by 16 (c) Division by 2 (d) Multiplication by 16
8. The 8421 BCD representation of decimal 59 is: (a) 00110101 (b) 00111011 (c) 01011100 (d) 01011001
9. A binary number occupies 20 bits. How many hexadecimal digits are needed to represent it exactly? (a) 4 (b) 6 (c) 8 (d) 5
10. A processor receives the hexadecimal value 7D3A. What is its decimal equivalent? (a) 32090 (b) 32058 (c) 32042 (d) 32122
11. A forensic analyst discovers that a malware author stored the value 111010111011 (binary). What is its hexadecimal equivalent? (a) EBB (b) EBB16 (c) EAB (d) EBB and EBB16 are equivalent
12. Which code was specifically designed to simplify decimal arithmetic operations? (a) Unicode (b) ASCII (c) EBCDIC (d) Excess-3
13. The Excess-3 code for decimal 9 is: (a) 1001 (b) 1010 (c) 1100 (d) 1111
14. The hexadecimal value 2A7F corresponds to which binary representation? (a) 0010101001111111 (b) 1010101111111111 (c) 1010100011111111 (d) 0010101001111110
15. Excess-3 code is: (a) Non-weighted (b) Weighted (c) ASCII-based (d) Floating-point
16. The Excess-3 code 0110 corresponds to decimal: (a) 6 (b) 9 (c) 12 (d) 3

ANSWERS

ANSWERS

1. **c** BCD is Binary Coded Decimal. 2. **b** 255 is FF in hex. 3. **a** the 1's complement of an Excess-3 digit gives its 9's complement. 4. **b** one hex digit maps to four bits. 5. **b** Unicode encodes many world languages. 6. **c** vector graphics scale without quality loss. 7. **c** a right shift of one bit divides by 2. 8. **d** 5 is 0101 and 9 is 1001, giving 01011001. 9. **d** 20 divided by 4 is 5 hex digits. 10. **b** 7D3A is 28672 plus 3328 plus 48 plus 10, which is 32058. 11. **d** the bits group in fours to E, B, B, so both forms name the same value. 12. **d** Excess-3 was built to simplify decimal arithmetic. 13. **c** 9 plus 3 is 12, which is 1100. 14. **a** 2A7F expands to 0010 1010 0111 1111. 15. **a** Excess-3 is a non-weighted code. 16. **d** 0110 is 6, minus the excess 3 gives 3.

Signed Numbers and Complements

1. What is the range of values representable by a 10-bit unsigned integer? (a) -1023 to 1023 (b) 0 to 1023 (c) -512 to 511 (d) 0 to 1024
2. The 1's complement representation 11100011 corresponds to: (a) -29 (b) -28 (c) -31 (d) -30
3. Which statement about one's complement is TRUE? (a) It has one representation of zero (b) Overflow never occurs (c) It has positive and negative zero (d) It has no negative numbers
4. In one's complement representation, the additive inverse of a number is obtained by: (a) Shifting right (b) Flipping all bits (c) Adding 1 (d) Shifting left
5. For an 8-bit sign-magnitude system, the decimal value -0 is represented by: (a) 11111111 (b) 00000000 (c) 01111111 (d) 10000000
6. An 8-bit unsigned register stores 11111111. What decimal value does it represent? (a) 127 (b) 255 (c) 254 (d) 128
7. An 8-bit register contains 10000000 in two's complement. What decimal value does it represent? (a) -127 (b) +128 (c) -128 (d) 0
8. How many representations of zero exist in sign-magnitude representation? (a) 2 (b) Depends on word length (c) 1 (d) 4
9. The two's complement of 00101100 is: (a) 10110100 (b) 11110011 (c) 11010011 (d) 11010100
10. Overflow occurs when: (a) Result exceeds representable range (b) Division by zero occurs (c) Sign changes unexpectedly (d) Bits are shifted
11. Which representation requires an end-around carry during addition? (a) Excess-3 (b) Sign Magnitude (c) One's Complement (d) Two's Complement
12. In sign-magnitude representation, the 8-bit value 10101010 represents: (a) +170 (b) -170 (c) +42 (d) -42
13. Why is two's complement widely used in modern computers? (a) Easier decimal conversion (b) Eliminates overflow (c) Greater storage efficiency (d) Faster and simpler hardware implementation
14. The major hardware disadvantage of sign-magnitude representation is: (a) Inability to represent positive numbers (b) Slow and complex arithmetic circuitry (c) Large memory usage (d) Limited range

ANSWERS

ANSWERS

1. **b** an n-bit unsigned range is 0 to $2^n - 1$, which is 0 to 1023. 2. **b** the sign bit is 1, invert to 00011100 which is 28, so the value is -28. 3. **c** one's complement has both a positive and a negative zero. 4. **b** flipping every bit negates a one's complement number. 5. **d** sign bit 1 with zero magnitude gives 10000000. 6. **b** all eight ones is 255. 7. **c** 10000000 in two's complement is -128. 8. **a** sign-magnitude has both +0 and -0. 9. **d** invert to 11010011 and add 1 to get 11010100. 10. **a** overflow is a result outside the representable range. 11. **c** one's complement addition needs an end-around carry. 12. **d** sign bit 1 with magnitude 0101010 which is 42, so -42. 13. **d** two's complement gives faster and simpler arithmetic hardware. 14. **b** sign-magnitude needs slow and complex arithmetic circuitry.

Floating Point

1. Which strategy helps reduce overflow/underflow occurrences? (a) Using ASCII (b) Using double precision arithmetic (c) Using fewer bits (d) Using BCD
2. A banking application depends heavily on accurate decimal calculations. Which architectural feature directly influences calculation accuracy? (a) Floating-point representation (b) Cache hierarchy (c) Control signals (d) Bus width
3. Approximately what numeric range can IEEE Double Precision represent? (a) $10^{\pm 38}$ (b) $10^{\pm 128}$ (c) $10^{\pm 308}$ (d) $10^{\pm 256}$
4. IEEE Double Precision uses: (a) 64 bits (b) 32 bits (c) 48 bits (d) 128 bits
5. A floating-point number is generally stored as: (a) Exponent, Sign, Mantissa (b) Mantissa, Exponent, Sign (c) ASCII digits (d) Sign, Mantissa, Exponent
6. What is the principal advantage of floating-point representation? (a) Exact decimal storage (b) Faster arithmetic (c) Elimination of overflow (d) Representation of very large and very small values
7. A weather simulation program requires highly accurate numerical computations. Which architectural component is most critical? (a) Floating-point representation (b) Cache replacement policy (c) Character encoding (d) Control bus design
8. Approximately what numeric range can IEEE Single Precision represent? (a) $10^{\pm 308}$ (b) $10^{\pm 38}$ (c) $10^{\pm 20}$ (d) $10^{\pm 10}$
9. IEEE Single Precision allocates how many bits to the exponent? (a) 8 (b) 23 (c) 7 (d) 11
10. The exponent bias used in IEEE Double Precision is: (a) 127 (b) 255 (c) 511 (d) 1023

ANSWERS

ANSWERS

1. **b** double precision adds bits, cutting overflow and underflow. 2. **a** floating-point representation sets numeric accuracy. 3. **c** IEEE double reaches about 10 to the power plus or minus 308. 4. **a** double precision is 64 bits. 5. **d** the exam takes Sign, Mantissa, Exponent (true IEEE field order is Sign, Exponent, Mantissa). 6. **d** floating point spans very large and very small values. 7. **a** floating-point representation drives simulation accuracy. 8. **b** IEEE single reaches about 10 to the power plus or minus 38. 9. **a** single precision uses 8 exponent bits. 10. **d** double precision uses a bias of 1023.

Instruction Types and Formats

1. An Instruction format defines: (a) Layout of bits within an instruction (b) Cache organization (c) Internal processor wiring (d) Operating system memory allocation
2. In a typical instruction format, the opcode field specifies: (a) Memory address (b) Operation to perform (c) Register contents (d) Interrupt priority
3. A processor designer wants to increase the number of available opcodes without changing instruction length. Which modification is most appropriate? (a) Increase RAM capacity (b) Increase opcode bits while reducing operand bits (c) Add pipeline stages (d) Increase cache size
4. Which instruction format generally results in fewer memory accesses? (a) Two-address (b) Three-address (c) Zero-address (d) One-address

5. A stack machine executes arithmetic operations without explicitly specifying operand addresses because operands are: (a) Stored in cache (b) Stored in virtual memory (c) Stored in ROM (d) Located at the top of the stack
6. A processor employs fixed-length 32-bit instructions. Which advantage is most likely achieved? (a) Faster instruction decoding (b) Elimination of registers (c) Unlimited opcode space (d) Smaller program size
7. An instruction contains the following fields: Opcode 8 bits, Register field 6 bits, Address field 18 bits. What is the total instruction length? (a) 40 bits (b) 32 bits (c) 36 bits (d) 30 bits
8. Which instruction format is most commonly associated with stack-based architectures? (a) Three-address (b) Zero-address (c) Two-address (d) One-address

ANSWERS

ANSWERS

1. **a** an instruction format is the layout of bits within an instruction. 2. **b** the opcode names the operation to perform. 3. **b** widen the opcode field and narrow the operand field. 4. **b** three-address instructions name all operands, cutting memory trips. 5. **d** a stack machine takes operands from the top of the stack. 6. **a** fixed length makes decoding faster. 7. **b** 8 plus 6 plus 18 is 32 bits. 8. **b** stack machines use zero-address instructions.

Addressing Modes

1. A compiler embeds a constant value directly within an instruction instead of storing it in memory. Which addressing mode is being used? (a) Immediate Addressing (b) Direct Addressing (c) Indirect Addressing (d) Register Addressing
2. Assume $R1 = 2000$ and $\text{Memory}[2000] = 500$. Using Register Indirect Addressing, the operand is: (a) Memory address only (b) 2000 (c) 500 (d) $R1$
3. Suppose the instruction address = 500 and displacement = +30. Using PC-relative addressing, the effective address becomes: (a) 30 (b) 500 (c) 470 (d) 530
4. Which addressing mode generally requires the fewest memory accesses during instruction execution? (a) Immediate (b) Indexed (c) Direct (d) Indirect
5. Consider $R2 = 300$, $\text{Memory}[320] = 900$, and instruction displacement = 20. Using Indexed Addressing with $R2$ as the index register, the effective address is: (a) 300 (b) 320 (c) 280 (d) 900
6. Given that the instruction specifies address = 700, $\text{Memory}[700] = 1500$, and $\text{Memory}[1500] = 60$. Under Indirect Addressing, the operand equals: (a) 700 (b) 60 (c) Undefined (d) 1500
7. An instruction contains the operand itself within the instruction. Which addressing mode is used? (a) Indexed (b) Register (c) Immediate (d) Direct
8. Suppose the base register = 4000 and displacement = 120. Using Base Register Addressing, the effective address is: (a) 3880 (b) 120 (c) 4000 (d) 4120
9. A processor designer aims to improve the efficiency of accessing dynamically allocated data structures such as linked lists. Which addressing mode is generally the most suitable? (a) Immediate addressing (b) Implied addressing (c) Register addressing (d) Indirect addressing

ANSWERS

ANSWERS

1. **a** a constant inside the instruction is immediate addressing. 2. **c** R1 points to address 2000, whose contents 500 are the operand. 3. **d** 500 plus 30 is 530. 4. **a** immediate holds the operand in the instruction, so no extra memory access. 5. **b** displacement 20 plus index 300 is 320. 6. **b** address 700 holds 1500, whose contents 60 are the operand. 7. **c** the operand inside the instruction is immediate addressing. 8. **d** base 4000 plus displacement 120 is 4120. 9. **d** indirect addressing follows pointers, which suits linked lists.

Memory Hierarchy and Types

1. A processor's control unit is responsible for: (a) Performing arithmetic calculations (b) Storing application files (c) Coordinating instruction execution through control signals (d) Managing peripheral power supply
2. A software engineer optimizes a program by storing frequently used variables in CPU registers instead of RAM. The primary benefit is: (a) Improved memory capacity (b) Lower bus traffic only (c) Reduced storage cost (d) Faster execution speed
3. Which bus carries information identifying memory locations and I/O device locations? (a) Control Bus (b) Data Bus (c) Memory Bus (d) Address Bus
4. An autonomous robot continuously performs sensor processing and actuator control. Which organizational component primarily coordinates these activities? (a) Control Unit (b) ALU (c) Register Set (d) Cache Memory
5. A motherboard manufacturer introduces a faster bus system, enabling quicker communication among components. This enhancement belongs to: (a) Computer Architecture (b) Instruction Design (c) Data Representation (d) Computer Organization
6. Which statement regarding ROM is correct? (a) ROM is mainly used for temporary program execution. (b) ROM stores firmware permanently. (c) ROM requires periodic refreshing. (d) ROM loses contents after power failure.
7. Why is cache memory generally constructed using SRAM rather than DRAM? (a) SRAM offers faster access without refresh operations. (b) SRAM is cheaper. (c) SRAM is non-volatile. (d) SRAM has larger storage capacity.
8. Which architectural feature most directly contributes to reducing the average instruction execution time without increasing clock frequency? (a) Increasing monitor resolution (b) Increasing hard disk capacity (c) Expanding ROM size (d) Increasing cache hit rate
9. Which of the following correctly orders memory from fastest to slowest? (a) Cache → Registers → SSD → RAM (b) Registers → Cache → Main Memory → Secondary Storage (c) Registers → Main Memory → Cache → SSD (d) Cache → Registers → Main Memory → Hard Disk
10. Which combination correctly matches memory type with volatility? (a) Flash Memory, Non-volatile (b) DRAM, Non-volatile (c) SRAM, Non-volatile (d) Cache Memory, Non-volatile
11. Which statement correctly explains memory hierarchy? (a) Main memory is faster than registers. (b) Every memory level has identical speed. (c) Higher levels are

faster, smaller, and more expensive per bit. (d) Lower levels are faster than upper levels.

12. A computer architect argues that registers should be larger than cache memory. Why is this argument incorrect? (a) Registers are virtual memory components. (b) Registers are intended for only the most immediate processor operations. (c) Registers belong to secondary storage. (d) Registers are slower.
13. Which memory characteristic is most critical for real-time control systems? (a) Low manufacturing cost (b) Predictable low access time (c) Storage capacity (d) High density
14. Why is secondary storage unsuitable for direct CPU execution of instructions? (a) It stores binary data. (b) It has significantly higher access latency than primary memory. (c) It is non-volatile. (d) It uses logical addressing.
15. Which statement best summarizes the fundamental trade-off underlying modern computer memory hierarchy? (a) All memory technologies have similar speed and cost. (b) Slower memories always consume more power than faster memories. (c) Faster memories generally provide larger capacities at lower cost. (d) Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.
16. Which statement best distinguishes SRAM from DRAM? (a) SRAM stores data using flip-flops whereas DRAM stores data using capacitors. (b) DRAM is faster than SRAM. (c) SRAM requires periodic refreshing while DRAM does not. (d) DRAM is non-volatile.
17. A computer engineer claims that increasing cache memory always eliminates processor-memory performance bottlenecks. Which statement best evaluates this claim? (a) It is correct only when DRAM is removed. (b) It is incorrect because cache cannot eliminate all memory access delays. (c) It is correct because cache replaces main memory. (d) It is always correct because cache operates at processor speed.
18. A processor spends considerable time waiting for data despite having a large cache. Which phenomenon is primarily responsible? (a) Address aliasing (b) Register spilling (c) Memory wall (d) Bus inversion
19. Which component stores the address of the memory location currently being accessed? (a) Memory Address Register (b) Memory Data Register (c) Program Counter (d) Instruction Register
20. A processor repeatedly accesses the same instruction during program execution. The improved execution speed primarily results from which memory hierarchy principle? (a) Spatial locality only (b) Temporal locality (c) Virtual addressing (d) Sequential consistency
21. A memory technology provides higher density but lower speed than SRAM. Which technology is being described? (a) ROM (b) EEPROM (c) Register memory (d) DRAM
22. Which memory level has the highest cost per bit? (a) SSD (b) Registers (c) Main memory (d) Hard disk
23. Which statement correctly describes virtual memory? (a) It replaces cache memory. (b) It physically increases RAM capacity. (c) It eliminates page faults. (d) It allows programs to execute using an address space larger than physical memory.
24. Which memory characteristic primarily determines how much information can be stored in a memory device? (a) Access time (b) Capacity (c) Transfer rate (d)

Volatility

25. A memory module has an access time of 60 ns and a cycle time of 90 ns. Which interpretation is correct? (a) Access time is longer than cycle time. (b) Cycle time has no practical significance. (c) Consecutive accesses must be separated by at least 90 ns. (d) A new memory operation can begin every 60 ns.
26. A workload frequently accesses neighboring memory locations after accessing a particular location. Which locality principle is being demonstrated? (a) Virtual locality (b) Random locality (c) Temporal locality (d) Spatial locality

ANSWERS

ANSWERS

1. **c** the control unit coordinates execution through control signals. 2. **d** registers are far faster than RAM, so execution speeds up. 3. **d** the address bus carries location information. 4. **a** the control unit coordinates the sensing and actuation. 5. **d** bus speed is a computer organization matter. 6. **b** ROM holds firmware permanently. 7. **a** SRAM is faster and needs no refresh. 8. **d** a higher cache hit rate cuts average access time. 9. **b** registers, then cache, then main memory, then secondary storage. 10. **a** flash memory keeps data without power, so it is non-volatile. 11. **c** higher levels are faster, smaller and costlier per bit. 12. **b** registers exist only for the most immediate operations, so they stay small. 13. **b** real-time control needs predictable low access time. 14. **b** secondary storage has far higher latency than primary memory. 15. **d** faster memory is smaller and costlier per bit, slower memory is larger and cheaper. 16. **a** SRAM uses flip-flops, DRAM uses capacitors. 17. **b** cache cannot remove every memory access delay. 18. **c** the processor-memory speed gap is the memory wall. 19. **a** the Memory Address Register holds the current address. 20. **b** reusing the same location is temporal locality. 21. **d** DRAM is denser and slower than SRAM. 22. **b** registers cost the most per bit. 23. **d** virtual memory gives an address space larger than physical RAM. 24. **b** capacity sets how much can be stored. 25. **c** the 90 ns cycle time sets the minimum gap between accesses. 26. **d** accessing nearby locations is spatial locality.

CPU Performance

1. A computer architect argues that processor performance should no longer be evaluated solely by clock frequency. Which of the following provides the strongest justification for this argument? (a) Modern processors execute exactly one instruction per clock cycle. (b) Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency. (c) Clock frequency completely determines instruction execution time. (d) Higher clock frequency eliminates cache misses.
2. A processor executes a benchmark requiring 12 billion instructions, operating at 4 GHz, with an average CPI of 1.5. What is the approximate execution time? (a) 2.5 seconds (b) 4.5 seconds (c) 5.5 seconds (d) 3.5 seconds
3. Suppose Processor X executes fewer instructions than Processor Y for the same program, but Processor Y completes execution faster due to better pipelining. Which statement is most accurate? (a) Processor X has a superior ISA. (b) Microarchitecture significantly affects performance. (c) ISA alone determines performance. (d) Instruction count is irrelevant.

ANSWERS

ANSWERS

1. **b** performance also depends on CPI, ILP, cache, branch prediction and latency. 2. **b** 12 billion times 1.5 divided by 4 billion is 4.5 seconds. 3. **b** better pipelining shows microarchitecture drives performance.

Pipelining, Cache and RISC/CISC

1. A processor implements speculative execution and branch prediction. The primary objective of these techniques is to: (a) Increase storage capacity. (b) Reduce pipeline stalls caused by control hazards. (c) Reduce transistor count. (d) Replace cache memory.
2. Variable-length instruction formats are commonly associated with: (a) CISC architectures (b) SIMD processors (c) RISC architectures (d) Harvard architecture
3. An embedded processor minimizes silicon area and power consumption by reducing instruction complexity and relying heavily on compiler optimization. This design reflects: (a) SIMD architecture (b) RISC philosophy (c) VLIW architecture (d) CISC philosophy
4. A computer architect wants to reduce compiler complexity by designing instructions that closely resemble high-level programming constructs. Which ISA philosophy best supports this objective? (a) Stack-based ISA (b) Harvard Architecture (c) RISC (d) CISC

ANSWERS

ANSWERS

1. **b** they reduce pipeline stalls from control hazards. 2. **a** variable-length instructions are a CISC trait. 3. **b** simple instructions plus heavy compiler reliance is the RISC philosophy. 4. **d** instructions resembling high-level constructs are CISC.

Von Neumann and Harvard, and the Rest

1. For a monochrome bitmap image, each pixel requires: (a) 2 bits (b) 8 bits (c) 1 bit (d) 4 bits
2. Photographs are generally better represented using: (a) Vector graphics (b) ASCII encoding (c) Excess-3 encoding (d) Bitmap graphics
3. A processor fetches an instruction while simultaneously reading data from memory. Which architecture makes this possible? (a) Stack Architecture (b) CISC Architecture (c) Von Neumann Architecture (d) Harvard Architecture
4. An embedded system uses separate memory modules for instructions and data to improve throughput. Which architectural design is being used? (a) Von Neumann Architecture (b) Stack Architecture (c) Pipeline Architecture (d) Harvard Architecture
5. Which characteristic is a major advantage of the Harvard Architecture? (a) Single memory space for all data (b) Simpler hardware implementation (c) Simultaneous access to instructions and data (d) Reduced memory requirements

ANSWERS

ANSWERS

1. **c** a monochrome pixel is one bit.
2. **d** photographs suit bitmap graphics.
3. **d** Harvard has separate instruction and data memories, allowing simultaneous access.
4. **d** separate instruction and data memories is the Harvard design.
5. **c** Harvard allows simultaneous access to instructions and data.

PROVE IT

The 2025 to 2026 Paper, Your Final Mock

The real paper, printed word for word, the one you are aiming at. Sit it Monday under exam conditions: two hours, answer Question One and any other two. Cover the worked answer, attempt the question, then check. This is the hall.

TEACH

HOW TO SIT IT

Question One is compulsory, 30 marks, and is where the numbers pile up. Then choose **any two of Questions Two to Five**, 20 marks each. Every answer is laid out the way you should write it on paper: the given values, the formula, the working stacked vertically, then the answer.

Question One (30 marks, compulsory)

AS PRINTED

25/26 Q1(A)

THE EXAMINER'S WORDS, UNEDITED

- a. i) Computer Systems are designed using two closely related but distinct concepts which are Computer Architecture and Computer Organization. Differentiate between these concepts.

2mks

- ii) Compare Von Neumann Architecture and Harvard Architecture. Include a diagram for each.

3mks

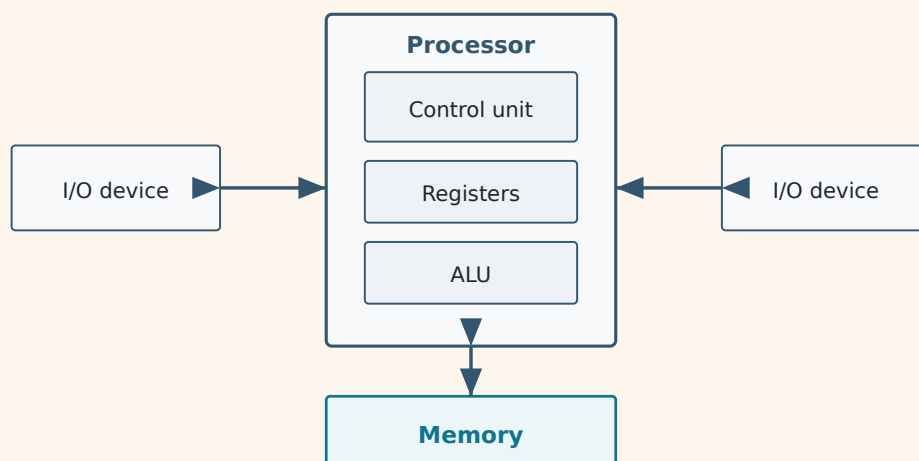
i) Differentiate computer architecture from computer organization.

Computer architecture is defined as the functional design and logical structure of the computer system as the programmer sees it: the instruction set, the data types, the addressing modes and the register organization. It answers the question "*What does the computer do?*" and serves as the blueprint for the system.

Computer organization is defined as the operational units and their interconnections that realize (carry out) that architecture: the control signals, the buses, the memory technology used and the cache. It answers the question "*How does the computer do it?*" A useful likeness: architecture is the building's design, organization is the actual construction of the building.

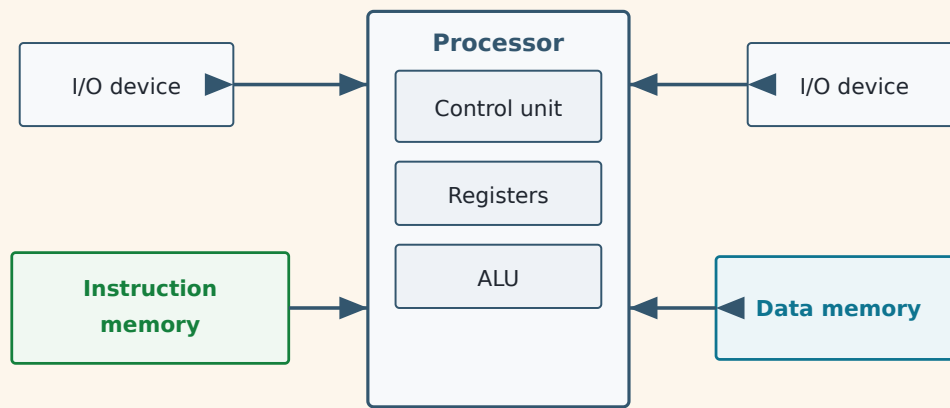
ii) Compare Von Neumann and Harvard architecture, with a diagram for each.

Von Neumann architecture is defined as a design that uses a **single** (shared) area of memory, reached over **one** bus, for both the program instructions and the data those instructions act on. It is simple to build, but the shared path is a bottleneck (an instruction fetch and a data access cannot happen at the same instant).



Von Neumann architecture, the layout the lecturer draws: one Memory serves the whole processor over a single shared path (instructions and data together).

Harvard architecture is defined as a design that uses **separate** memories and **separate** buses for instructions and data, which lets it **fetch an instruction and read data at the same time** (a form of parallelism), at the cost of duplicating the address, data and control lines.



Harvard architecture, the lecturer's layout: a separate instruction memory (fetched, one arrow in) and data memory (read and written), each on its own path, so both are reached at once.

AS PRINTED

25/26 Q1(B)

THE EXAMINER'S WORDS, UNEDITED

b. A computer manufacturer is evaluating whether to replace a non-pipelined processor with a five-stage pipelined processor. The processor executes the following instruction mix: Arithmetic – 40%, Memory Access – 35%, Branch – 20%, Floating Point – 5%. The Non-pipelined processor clock cycle is 25ns. Pipelined processor stage times are: IF – 5ns, ID – 4ns, EX – 6ns, MEM – 5ns and WB – 4ns. Furthermore, Pipeline register overhead is 1 ns. Branch instructions incur a two-cycle penalty, Floating-point instructions has three cycles for EX stage, Memory accesses experience cache misses 10% of the time, adding four extra cycles. The program contains 20 million instructions.

- i) Determine the pipelined clock cycle. **1mk**
- ii) Calculate the actual CPI. **2mks**
- iii) Determine the actual execution time. **1mk**
- iv) Compute the speedup over the non-pipelined processor. **1mk**
- v) Explain why the speedup differs from the theoretical maximum. **1mk**

GIVEN	mix arithmetic 40%, memory 35%, branch 20%, floating point 5% ; five stage times 5, 4, 6, 5, 4 ns ; register overhead 1 ns ; non-pipelined clock 25 ns ; IC = 20 000 000 ; branch adds a 2-cycle penalty, floating point adds 4 extra cycles, a memory access misses 10% of the time adding 4 more.
FIND	pipelined clock, actual CPI, execution time, speedup, and why it is short of the max
FORMULA	clock = slowest stage + overhead · actual CPI = $1 + \sum(\text{fraction} \times \text{extra cycles})$ · time = IC × CPI × T · speedup = time(non-pipe) / time(pipe)

The stem is dense and does not say whether each penalty is extra or total, so state the reading: base CPI 1; branch +2; memory +(2 + 0.10×4) = 2.4; floating point +4; arithmetic +0.

```

Clock  = max(5,4,6,5,4) + overhead
        = 6 + 1 = 7 ns

CPI     = 1                      (base, one per cycle)
        + 0.20 × 2                (branch)
        + 0.35 × 2.4              (memory: 2 + 0.10×4)
        + 0.05 × 4                (floating point)
        + 0.40 × 0                (arithmetic)
        = 1 + 0.40 + 0.84 + 0.20
        = 2.44

Time    = IC × CPI × T
        = 20,000,000 × 2.44 × 7 ns = 0.3416 s
Non-pipe = 20,000,000 × 25 ns    = 0.5 s
Speedup  = 0.5 / 0.3416          = 1.46

```

Clock 7 ns; CPI 2.44; time 0.3416 s; speedup 1.46. (v) It is far below the ideal 5 because the branch, floating-point and cache-miss penalties and the register overhead all raise the CPI above 1.

THE EXAMINER'S WORDS, UNEDITED

- c. A spacecraft telemetry system uses a 16-bit processor to transmit environmental measurements. The following values are to be stored - Temperature = -182; Pressure = 487; Altitude = -1023. The processor uses 2's complement representation for signed integers.
- Determine the range of values representable in a 16-bit two's complement system. Indicate whether the given numbers fall within the range of value **2mks**
 - Represent each value in: Binary and Hexadecimal **3mks**
 - Demonstrate how the processor uses two's complement to compute **2mks**
 $(-182) + 487$

GIVEN 16-bit two's complement ; Temperature -182 , Pressure 487 , Altitude -1023
FIND the range and whether each fits ; each in binary and hex ; $(-182) + 487$
FORMULA range = -2^{15} to $2^{15} - 1$ · negative = invert the positive and add 1 · group 4 bits per hex digit

Range: -2^{15} to $2^{15} - 1 = -32768$ to 32767 (all three fit)

$487 = 0000\ 0001\ 1110\ 0111 = 01E7$

-182 : $+182 = 0000\ 0000\ 1011\ 0110$
 invert = $1111\ 1111\ 0100\ 1001$
 $+ 1 = 1111\ 1111\ 0100\ 1010 = FF4A$

-1023 : $+1023 = 0000\ 0011\ 1111\ 1111$
 invert = $1111\ 1100\ 0000\ 0000$
 $+ 1 = 1111\ 1100\ 0000\ 0001 = FC01$

$(-182) + 487$:

$1111\ 1111\ 0100\ 1010$

$+ 0000\ 0001\ 1110\ 0111$

$1\ 0000\ 0001\ 0011\ 0001$ drop the carry off the top

$0000\ 0001\ 0011\ 0001 = 256+32+16+1 = 305$

Range -32768 to 32767 (all fit). $487 = 01E7$, $-182 = FF4A$, $-1023 = FC01$. $(-182) + 487 = 305$: the machine adds the patterns and gets the right signed answer.

THE EXAMINER'S WORDS, UNEDITED

- d. A processor executes one million(1M) instructions distributed as follows.

Addressing Mode	Percentage	Cycles
Register	40%	1
Immediate	20%	1
Direct	15%	2
Indirect	10%	4
Indexed	10%	3
Relative	5%	2

- Compute the average CPI **2mks**
- Determine total clock cycles. **1mk**
- If indirect addressing is optimized to 2 cycles, compute the new CPI. **1mk**
- Evaluate the performance improvement. **2mks**

GIVEN 1,000,000 instructions ; per-mode share and cycles as tabled
 FIND average CPI, total cycles, new CPI after optimising indirect, improvement
 FORMULA $CPI = \Sigma(\text{fraction} \times \text{cycles}) \cdot \text{total} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new})/\text{old}$

$$\begin{aligned} \text{Avg CPI} &= 0.40 \times 1 + 0.20 \times 1 + 0.15 \times 2 + 0.10 \times 4 + 0.10 \times 3 + 0.05 \times 2 \\ &= 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10 \\ &= 1.70 \end{aligned}$$

$$\text{Total cycles} = 1,000,000 \times 1.70 = 1,700,000$$

$$\begin{aligned} \text{Indirect 4} \rightarrow 2: \text{ new CPI} &= 1.70 - 0.10 \times 4 + 0.10 \times 2 \\ &= 1.70 - 0.20 = 1.50 \end{aligned}$$

$$\text{Improvement} = (1.70 - 1.50) / 1.70 = 0.20 / 1.70 = 11.76\%$$

CPI 1.70; total cycles 1,700,000; new CPI 1.50; improvement 11.76%.

THE EXAMINER'S WORDS, UNEDITED

e. A processor contains the following information: PC = 7000; R1 = 1500;

Memory Contents

Address	Value
7000	1200
1200	4500
1500	6000
3200	8100
4500	999
6000	200
8100	400

Assume an instruction contains address field 1200.

Determine the Effective Address (EA) and operand for the following addressing modes:

- i) Immediate addressing mode **1mk**
- ii) Direct addressing mode **1mk**
- iii) Indirect addressing mode **1mk**
- iv) Register addressing mode **1mk**
- v) Register Indirect (R1) addressing mode **1mk**
- vi) Relative addressing mode **1mk**

GIVEN PC = 7000, R1 = 1500, address field = 1200 ; M[1200]=4500, M[4500]=999, M[1500]=6000

FIND the EA and operand under each mode

FORMULA the mode says how to turn the field into the EA ; the operand is M[EA]

MODE	EFFECTIVE ADDRESS	OPERAND
i) Immediate	none	the field = 1200
ii) Direct	1200	M[1200] = 4500
iii) Indirect	M[1200] = 4500	M[4500] = 999
iv) Register	none	R1 = 1500
v) Register indirect	R1 = 1500	M[1500] = 6000
vi) Relative	PC + 1200 = 8200	M[8200], not in the given table

Immediate 1200; Direct 4500; Indirect 999; Register 1500; Register indirect 6000; Relative EA 8200 (its operand is not listed).

Question Two (20 marks)

THE EXAMINER'S WORDS, UNEDITED

- a. Assume a computer has 32-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache (decimal value) would you look for each of the following addresses? Addresses are given in hexadecimal for convenience. **4mks**
- CA2DFC12
 - F24F25FF
 - 24BBABCD
 - C1D9EEF2

GIVEN 32-bit address ; 16 words per block ; direct-mapped, 256 blocks
 FIND the cache line each hex address maps to, in decimal
 FORMULA $\text{offset} = \log_2(\text{words/block})$; $\text{index} = \log_2(\text{lines})$; the index field IS the line

$\text{offset} = \log_2(16) = 4 \text{ bits}$ (one hex digit)
 $\text{index} = \log_2(256) = 8 \text{ bits}$ (two hex digits)
 So: drop the last hex digit (offset), read the next two (the line):

CA2DFC1|2 -> C1 = 193
 F24F25F|F -> 5F = 95
 24BBABC|D -> BC = 188
 C1D9EEF|2 -> EF = 239

Lines: CA2DFC12 → 193, F24F25FF → 95, 24BBABCD → 188, C1D9EEF2 → 239.

THE EXAMINER'S WORDS, UNEDITED

b. Consider the following code segment executed on a classic five-stage RISC processor.

I1: LW R1,0(R2)
 I2: ADD R3,R1,R4
 I3: SUB R5,R3,R6
 I4: SW R5,8(R2)
 I5: BEQ R5,R0,LABEL
 I6: MUL R7,R5,R8

Assume:

- forwarding is available
- branch resolved in EX stage
- load-use hazards require one stall
- multiplier occupies EX for three cycles
- separate instruction and data caches

- i) Identify and explain every pipeline hazard. **5mks**
 ii) Draw the pipeline timing diagram **2mks**
 iii) Determine the total execution cycles **1mk**
 iv) Suggest any two hardware improvements that would reduce stalls. **1mk**

GIVEN	six instructions on a classic 5-stage RISC pipeline ; forwarding on ; branch resolved in EX ; load-use costs 1 stall ; MUL holds EX for 3 cycles ; separate caches
FIND	the hazards, the timing diagram, the total cycles, two improvements
FORMULA	ideal = $(k + n - 1)$ cycles, then add every stall: 1 per load-use, and $(EX \text{ cycles} - 1)$ per multi-cycle execute

Hazards

I1→I2 is a **load-use RAW** (R1 not ready until MEM, so I2 stalls one cycle). I2→I3, I3→I4, I3→I5, I3→I6 are ordinary RAW hazards **hidden by forwarding**. I6's **multiply holds EX for three cycles**, adding two extra at the end. Structural hazards are avoided by the separate caches.

Timing diagram (one stall, then the long multiply)

CYC	1	2	3	4	5	6	7	8	9	10	11	12	13
I1 LW	IF	ID	EX	ME	WB								
I2 ADD		IF	ID	--	EX	ME	WB						
I3 SUB			IF	--	ID	EX	ME	WB					
I4 SW					IF	ID	EX	ME	WB				
I5 BEQ						IF	ID	EX	ME	WB			
I6 MUL							IF	ID	EX	EX	EX	ME	WB

One load-use stall (the -- in cycle 4) plus two extra multiply cycles give a total of **13 cycles** (a clean run of 6 would be $5 + 6 - 1 = 10$). Two improvements: add more forwarding paths, and pipeline or duplicate the multiplier.

THE EXAMINER'S WORDS, UNEDITED

c. A processor has the following configuration: Main memory is 1 GB; Cache is 512 KB; Block size is 128 bytes and it is a 4-way Set Associative cache.

i) Determine the total number of cache blocks. **1mk**

ii) Determine the number of sets. **1mk**

iii) Construct the complete address format. **1mk**

iv) If the hit rate is 96%, cache access time is 2 ns, and main memory access time is 80 ns, calculate the Average Memory Access Time (AMAT). **2mks**

v) Discuss how increasing associativity beyond four ways might affect performance. **2mks**

GIVEN main memory 1 GB ; cache 512 KB ; block 128 B ; 4-way ; $h = 0.96$, $T_c = 2$ ns, $T_m = 80$ ns

FIND blocks, sets, the address format, the AMAT

FORMULA $\text{blocks} = \text{cache}/\text{block}$; $\text{sets} = \text{blocks}/\text{ways}$; $\text{tag} = \text{addr} - \text{set} - \text{offset}$; $\text{AMAT} = T_c + (1-h) \times T_m$

Blocks = $512 \text{ KB} / 128 \text{ B} = 2^{19} / 2^7 = 4096$
 Sets = $4096 / 4 = 1024$
 Address = $1 \text{ GB} = 2^{30} \rightarrow 30$ bits
 offset = $\log_2(128) = 7$
 set = $\log_2(1024) = 10$
 tag = $30 - 10 - 7 = 13$
 AMAT = $T_c + (1-h)T_m = 2 + 0.04 \times 80 = 2 + 3.2 = 5.2$ ns

4096 blocks; 1024 sets; address tag 13 / set 10 / offset 7; AMAT 5.2 ns. (v) More ways cuts conflict misses but needs more tag comparators, so beyond a point it adds cost and delay for little gain.

Question Three (20 marks)

THE EXAMINER'S WORDS, UNEDITED

- a. A computer system has the following characteristics:
- Main memory = 64 MB
 - Cache memory = 128 KB
 - Block size = 64 bytes
- i) Determine the number of cache lines **1½mks**
- ii) For Direct Mapping, determine: Number of index bits; Number of tag bits; Number of offset bits **1½mks**
- iii) For a 4-way Set Associative Cache, determine: Number of sets; Number of set index bits; Number of tag bits **1½mks**
- iv) Explain why fully associative mapping eliminates conflict misses but introduces another performance challenge. **1½mks**

GIVEN main memory 64 MB ; cache 128 KB ; block 64 B
 FIND lines ; direct fields ; 4-way fields ; the fully-associative trade-off
 FORMULA lines = cache/block ; offset = $\log_2(\text{block})$; tag = addr – index(or set) – offset

Lines = 128 KB / 64 B = $2^{17} / 2^6 = 2^{11} = 2048$
 Address = 64 MB = 2^{26} -> 26 bits ; offset = $\log_2(64) = 6$

Direct: index = $\log_2(2048) = 11$; tag = 26 - 11 - 6 = 9
 4-way : sets = 2048/4 = 512 ; set = $\log_2(512) = 9$; tag = 26 - 9 - 6 = 11

2048 lines. Direct: index 11, tag 9, offset 6. 4-way: set 9, tag 11, offset 6.
 (iv) Fully associative lets any block go in any line, so no conflict misses, but every line's tag must be searched, which is slow and costly.

THE EXAMINER'S WORDS, UNEDITED

b. A 5-stage pipelined processor has instruction fetch (IF), Instruction decode (ID), Operand Fetch (OF), Execute (Ex) and Write Back (WB) stages. The IF, ID, OF and WB stages take 1 clock cycle each for any instruction. The PO stage take 1 cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline.

Draw the cycle diagram and determine the number of cycles needed to execute the following sequence of instructions:

- i. When Operand forwarding is used **4½mks**
- ii. When Operand forwarding is not used **5½mks**

I1 MUL R2, R0, R1 ; R2 ← R0 X R1
 I2 DIV R5, R3, R4 ; R5 ← R3/R4
 I3 ADD R2, R5, R2 ; R2 ← R5 + R2
 I4 SUB R5, R2, R6 ; R5 ← R2 - R6

GIVEN	5 stages IF, ID, OF, EX, WB, one cycle each except EX = 1 (ADD/SUB), 3 (MUL), 6 (DIV) ; four instructions ; do it with and without forwarding
FIND	the cycle diagram and total cycles for each case
FORMULA	no single formula once EX is multi-cycle: lay the schedule out cycle by cycle, a long EX blocks those behind it, then read the total off the last write-back

The paper prints "PO stage", struck through in pen to EX. Instructions: I1 MUL R2,R0,R1 ; I2 DIV R5,R3,R4 ; I3 ADD R2,R5,R2 (needs R5 from I2 and R2 from I1) ; I4 SUB R5,R2,R6 (needs R2 from I3). Read each cell as the stage that instruction is in that cycle; "--" is a stall.

(i) With operand forwarding

INSTR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I1 MUL	IF	ID	OF	EX	EX	EX	WB								
I2 DIV		IF	ID	OF	--	--	EX	EX	EX	EX	EX	EX	WB		
I3 ADD			IF	ID	--	--	OF	--	--	--	--	--	EX	WB	
I4 SUB				IF	--	--	ID	--	--	--	--	--	OF	EX	WB

The DIV holds EX for six cycles (7 to 12) and blocks everything behind it. With forwarding, I3 takes R5 straight from I2's EX and runs at cycle 13, and I4 takes R2 from I3 and runs at 14, so the last write-back is at **cycle 15**.

(ii) Without operand forwarding

INSTR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
I1 MUL	IF	ID	OF	EX	EX	EX	WB										
I2 DIV		IF	ID	OF	--	--	EX	EX	EX	EX	EX	EX	WB				
I3 ADD			IF	ID	--	--	OF	--	--	--	--	--	--	EX	WB		
I4 SUB				IF	--	--	ID	--	--	--	--	--	--	--	OF	EX	WB

Without forwarding, I3 must wait for I2's **write-back** before it can read R5 (one cycle later, EX at 14), and I4 likewise waits for I3, so the last write-back slips to **cycle 17**.

Total = **15 cycles** with forwarding, **17 cycles** without. Forwarding saved 2 cycles here.

THE EXAMINER'S WORDS, UNEDITED

- c. i) State Amdahl's law' **2mks**
 ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system.
2mks

GIVEN cache 10 times faster than memory ; used 85% of the time
 FIND state Amdahl's law, then the overall speedup
 FORMULA $S = 1 / ((1-f) + f/s)$, with f the sped-up fraction and s its speedup

Amdahl's law states that the overall speedup obtainable by improving one part of a system is limited by the fraction of the total time that part is actually used: even if that part is made infinitely fast, the speedup can never exceed $1/(1-f)$, where f is the fraction of the time that is sped up.

$f = 0.85$ (cache hits), $s = 10$
 $S = 1 / ((1 - 0.85) + 0.85/10)$
 $= 1 / (0.15 + 0.085)$
 $= 1 / 0.235$
 $= 4.26$

$S = 4.26$, far below 10, because the 15% of slow accesses dominate.

Question Four (20 marks)

THE EXAMINER'S WORDS, UNEDITED

a. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format.

$X = C1400000h$, $Y = 42100000h$, $Z = 41400000h$

Convert X, Y and Z to decimal numbers and determine which of the following is true. **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

GIVEN $X = C1400000h$, $Y = 42100000h$, $Z = 41400000h$, IEEE-754 single precision
 FIND each in decimal, then which claim is true
 FORMULA $\text{value} = (-1)^s \times 1.M \times 2^{(E-127)}$, where s = sign bit, M = mantissa, E = exponent field

$X = C1400000 = 1\ 100000010\ 100\ 0000\dots$
 sign 1 (neg); exp 130, $e = 3$; $1.100\dots = 1.5$
 $X = -1.5 \times 2^3 = -12$

$Y = 42100000 = 0\ 10000100\ 001\ 0000\dots$
 sign 0; exp 132, $e = 5$; $1.001\dots = 1.125$
 $Y = 1.125 \times 2^5 = 36$

$Z = 41400000 = 0\ 10000010\ 100\ 0000\dots$
 sign 0; exp 130, $e = 3$; 1.5
 $Z = +1.5 \times 2^3 = 12$

$(Y - Z) = 36 - 12 = 24 > 0$ TRUE
 $X + Z = -12 + 12 = 0$ TRUE
 $Z = X + Y$? $12 = -12 + 36 = 24$ FALSE

$X = -12$, $Y = 36$, $Z = 12$. (i) true, (ii) true, (iii) false.

THE EXAMINER'S WORDS, UNEDITED

b. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required.

$X = ((A * B) + C/D)/(F * G + J * K)$ **8mks**

On the sheet the sentence "For each format determine the number of bytes required" is struck through in pen, so in that one sitting only the four programs were wanted. But the PRINTED paper asks for the byte counts too, so here is both: the programs, then the byte count for each format. If the strike is not on your paper, you have the full answer; if it is, the totals cost four lines and cannot lose marks.

Three-address (numerator, then denominator, then divide):

```
MUL R1, A, B      ; R1 = A*B
DIV R2, C, D      ; R2 = C/D
ADD R1, R1, R2    ; numerator
MUL R2, F, G
MUL R3, J, K
ADD R2, R2, R3    ; denominator
DIV X, R1, R2     ; X = numerator / denominator
```

Two-address: MOV/op with the destination doubling as a source. **One-address:** work through the accumulator (LOAD, op, STORE). **Zero-address:** a stack (PUSH the eight operands, then each ADD/MUL/DIV pops two and pushes the result, finally POP X).

Byte count for each format (the printed part, worked)

GIVEN the paper gives no field sizes, so state the usual convention: opcode = 1 byte, each address = 2 bytes

FORMULA bytes per instruction = opcode + (address fields × 2) · total = instruction count × bytes per instruction

FORMAT	INSTRUCTIONS	BYTES
3-address	7	$7 \times (1 + 3 \times 2) = 7 \times 7 = \mathbf{49}$
2-address	12	$12 \times (1 + 2 \times 2) = 12 \times 5 = \mathbf{60}$
1-address	17	$17 \times (1 + 1 \times 2) = 17 \times 3 = \mathbf{51}$
0-address	16	$8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = \mathbf{34}$

Same X in all four formats. Byte totals **49, 60, 51 and 34** for 3, 2, 1 and 0-address. Fewer address fields shortens each instruction but needs more of them, so the totals stay close, and 0-address is smallest here because its operation instructions carry no address at all.

THE EXAMINER'S WORDS, UNEDITED

c. Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte **3mks**

GIVEN 17.12 in by 9.45 in at 250 dpi ; 1 bit per pixel
 FIND the memory in megabytes
 FORMULA pixels = inches $\times$ dpi (both ways) ; bytes = pixels $\times$ bits / 8 ; MB = bytes / 1024 / 1024

pixels across = $17.12 \times 250 = 4280$
 pixels down = $9.45 \times 250 = 2362.5$
 total pixels = $4280 \times 2362.5 = 10,111,500$
 bytes = $10,111,500 / 8 = 1,263,937.5$
 MB = $1,263,937.5 / 1024 / 1024 = 1.21$ MB

$10,111,500$ pixels $\times$ 1 bit = $1,263,937.5$ bytes $\approx$ 1.21 MB. State the megabyte convention you used.

Question Five (20 marks)

THE EXAMINER'S WORDS, UNEDITED

- a. Represent the following -1240.56 using Single Precision IEEE format for floating point numbers. **6mks**

GIVEN -1240.56, single precision
 FIND the 32-bit pattern (and its hex)
 FORMULA sign ; normalise $|x|$ to $1.M \times 2^e$; exp field = $e + 127$; mantissa $M = f$ to 23 bits

sign = 1 (negative)
 1240.56 in binary = $1.0011011000100011110101... \times 2^{10}$
 exp field = $10 + 127 = 137 = 1000\ 1001$
 mantissa (23 bits) = 0011011 0001000 1111010 11
 pattern = 1 10001001 00110110001000111101011
 = C49B11EB (truncated; C49B11EC if rounded)

-1240.56 = **C49B11EB** (truncated), or C49B11EC rounded. State which.

THE EXAMINER'S WORDS, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- Pipeline cycle time **1mk**
 - Non-pipeline execution time **1mk**
 - Speed up ratio **1mk**
 - Pipeline time for 500 tasks **1mk**
 - Sequential time for 500 tasks **1mk**

IN THE HALL

25/26 Q5(B)

GIVEN 4 phases 60, 50, 90, 80 ns ; latch 10 ns ; 500 tasks
 FIND cycle time, one-task time, speedup, and the times for 500 tasks
 FORMULA $\text{cycle} = \text{slowest} + \text{latch}$; $\text{pipe}(n) = (k + n - 1) \times \text{cycle}$; $\text{seq}(n) = n \times \Sigma \text{phases}$;
 $\text{speedup} = \Sigma \text{phases} / \text{cycle}$

Cycle = $\max(60, 50, 90, 80) + 10 = 90 + 10 = 100 \text{ ns}$
 One task = $60 + 50 + 90 + 80 = 280 \text{ ns}$
 Speedup = $280 / 100 = 2.8$
 Pipe 500 = $(4 + 500 - 1) \times 100 = 503 \times 100 = 50,300 \text{ ns}$
 Seq 500 = $500 \times 280 = 140,000 \text{ ns}$

Cycle 100 ns; one task 280 ns; speedup 2.8; 500 tasks pipelined 50,300 ns vs sequential 140,000 ns.

AS PRINTED

25/26 Q5(C)

THE EXAMINER'S WORDS, UNEDITED

- c. Explain the following terms **3mks**
- Locality of reference
 - Temporal locality

IN THE HALL

25/26 Q5(C)

i) Locality of reference is defined as the tendency of a program to keep accessing the same small set of memory locations over a short period, a consequence of loops and subroutines; it is why a small fast cache can satisfy most accesses.

ii) Temporal (time) locality is defined as: if a memory location is accessed, it is very likely to be accessed **again soon** (the same instruction or variable reused). Its partner, **spatial (space) locality**, is defined as: if a location is accessed, the **nearby** addresses are very likely to be accessed next (as when stepping through an array).

THE EXAMINER'S WORDS, UNEDITED

- d. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E000h **6mks**

IN THE HALL

25/26 Q5(D)

GIVEN C4F2E000h, single precision
 FIND the decimal value
 FORMULA $\text{value} = (-1)^s \times 1.M \times 2^{(E - 127)}$, where s = sign bit, M = mantissa, E = exponent field

C4F2E000 = 1 10001001 111001011100000000000000
 sign 1 (neg); exp field 1000 1001 = 137, e = 137 - 127 = 10
 1.M = 1.1110010111 = 1.8974609375
 value = $-1.8974609375 \times 2^{10} = -1.8974609375 \times 1024$
 = -1943

C4F2E000h = -1943.

● MUST DO

THE NIGHT BEFORE

Redo Question One on blank paper from memory, then one cache split, one pipeline speedup, one CPI and one Amdahl. If those flow, you are ready. Walk in and start with Question One.